

Q&A DSMT

Capitolo 1 - Concorrenza e i benefici della parallelizzazione

Rappresentazione formale di un'esecuzione concorrente:

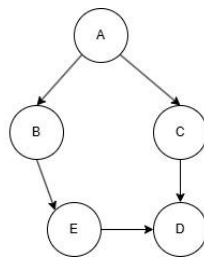
- L'insieme di azioni da compiere $\rightarrow A$.
- L'insieme dei vincoli di precedenza $\rightarrow PC$.

Definizione di concorrenza

Siano due azioni $a_i, a_j \in A$.

Queste due azioni sono dette **concorrenti** ($a_i || a_j$) **se e solo se** non ci sono relazioni del tipo $(a_i \leq_p a_j) \vee (a_j \leq_p a_i)$.

Attenzione: la relazione di concorrenza **non è transitiva**.



Sia il grafo in figura: C ed E sono concorrenti, così come B e C , però B ed E non sono concorrenti perché B deve precedere E ($B \leq E$).

Relazione di precedenza (aka vincolo di precedenza)

$a \rightarrow b$ significa che b viene per forza dopo a , in parole povere: b non può essere fatta senza che prima non sia fatta a .

L'ordine conta: $a \rightarrow b \neq b \rightarrow a$.

Se non esiste alcuna relazione del tipo $a \rightarrow b$ o $b \rightarrow a$, allora a e b sono concorrenti ($a||b$) e possono essere svolte in parallelo.

DAG - Directed Acyclic Graph

- **Diretto**: ogni arco ha una direzione (*freccia*).
- **Aciclico**: non esistono cicli, non puoi partire da un nodo, seguire una sequenza di archi e ritornare allo stesso nodo.

Ogni nodo rappresenta un'**unità di lavoro** (*azione*). Ogni arco rappresenta una relazione di precedenza tra due unità di lavoro.

Un'unità di lavoro è un insieme di operazioni che devono essere eseguite "*in blocco*" e non possono essere divise o eseguite in altro modo.

PC (Precedence Constraints)

Definiscono l'ordine in cui le unità di lavoro devono essere eseguite in un sistema.

Questo insieme di regole è indicato con $PC = \{ a \rightarrow b, b \rightarrow c, c \rightarrow d \}$

Partial Order

Si tratta di una relazione binaria $\leq$ su un insieme di azioni A .

Stabilisce un ordinamento tra alcuni elementi di A , rispettando la riflessività, antisimmetria e transitività.

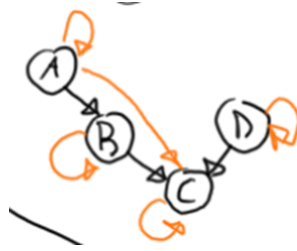
In un partial order, alcuni elementi possono rimanere non confrontabili tra loro.

Non-strict partial order ($\leq$):

- **Riflessività**: $a \leq a$
- **Antisimmetria**: Se $(a \leq b \wedge b \leq a) \Rightarrow a = b$.

Questo significa che se $(a \leq b \wedge b \neq a) \Rightarrow b \not\leq a$

- **Transitività**: Se $a \leq b \wedge b \leq c \Rightarrow a \leq c$



Strict partial order ($<$):

- **Antisimmetria:** Se $(a \leq b \wedge b \leq a) \Rightarrow a = b$.
 - Questo significa che se $(a \leq b \wedge b \neq a) \Rightarrow b \not\leq a$
- **Transitività:** Se $a \leq b \wedge b \leq c \Rightarrow a \leq c$

1. Irriflessività:

$$\neg(a < a) \quad (\text{cioè, non esiste alcun } a \text{ tale che } a < a)$$

Significa che nessun elemento può essere **minore di sé stesso**.

👉 **Esempio:** Se $<$ rappresenta "avviene prima" in un sistema concorrente, allora non ha senso dire che un'azione avviene prima di sé stessa.

Dunque devo togliere le frecce che rientrano nei nodi.

Elementi confrontabili

Una coppia di elementi (a, b) è **confrontabile** in un partial order **se esiste una relazione di ordine** tra loro $\rightarrow a \leq b$ oppure $b \leq a$.

Se non ne esiste **nessuna** $\rightarrow$ **Non confrontabili** $\rightarrow$ **Indipendenti**.

Transitive Closure - PC+

In un partial order esiste la transitività $\rightarrow$ esiste una chiusura transitiva.

Partendo dal PC è possibile definire la chiusura transitiva $\leq_p = PC^+$.

Come si crea una chiusura transitiva?

Se esistono $(a \rightarrow b)$ e $(b \rightarrow c)$, allora viene aggiunto anche $(a \rightarrow c)$.

$$PC = \{a \rightarrow b, b \rightarrow c\} \Rightarrow PC^+ = \{a \rightarrow b, b \rightarrow c, a \rightarrow c\}$$

POSET - Partial Order SET

Modello matematico con cui modelliamo una esecuzione concorrente.

Sia A l'insieme delle unità di lavoro nel sistema.

$$A + PC^+ = POSET \text{ (coppia)}$$

Ovvero un **POSET** rappresenta un insieme di **azioni** che seguono una relazione di **ordine parziale**.

A cosa serve il POSET?

Da esso possiamo capire quali sono tutte le unità di lavoro che devono essere eseguite prima una certa azione.

Diagramma di Hasse e riduzione transitiva

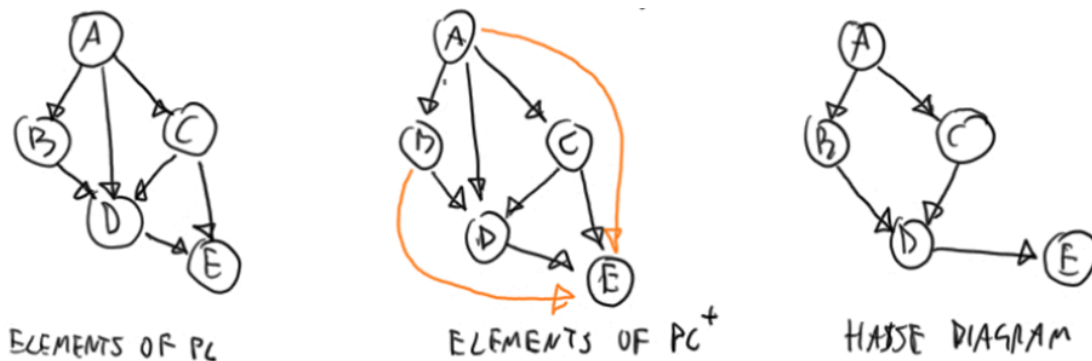
Rappresentazione grafica di un POSET $\rightarrow$ Diagramma di Hasse.

Come si costruisce il diagramma di Hasse? Dalla **riduzione transitiva** di PC .

Riduzione Transitiva $\rightarrow$ Il più piccolo set R tale che $R^+ \equiv PC^+$.

La riduzione transitiva è unica.

“Più piccolo” $\rightarrow$ Il grafo R con il **numero minimo di archi** ma con **transitive closure** R^+ **equivalente a quella di PC** , ovvero risultano equivalenti.



Perché usiamo la riduzione transitiva? Nel diag. di Hasse si **omettono gli archi ridondanti**.

Total Order

Caso particolare di partial order in cui **ogni coppia di elementi è confrontabile**.

$$\forall a_i, a_j \in A \Rightarrow (a_i \leq a_j) \vee (a_j \leq a_i)$$

Tutte le azioni sono due a due confrontabili $\rightarrow$ Nessuna azione è indipendente.

Se un sistema segue un **ordine totale**, allora per ogni coppia di azioni a e b , una deve necessariamente avvenire prima dell'altra.

Se invece il sistema segue un **ordine parziale**, esistono azioni che potrebbero non essere confrontabili, ovvero **possono avvenire indipendentemente l'una dall'altra**.

Catena e la nozione di processo/thread

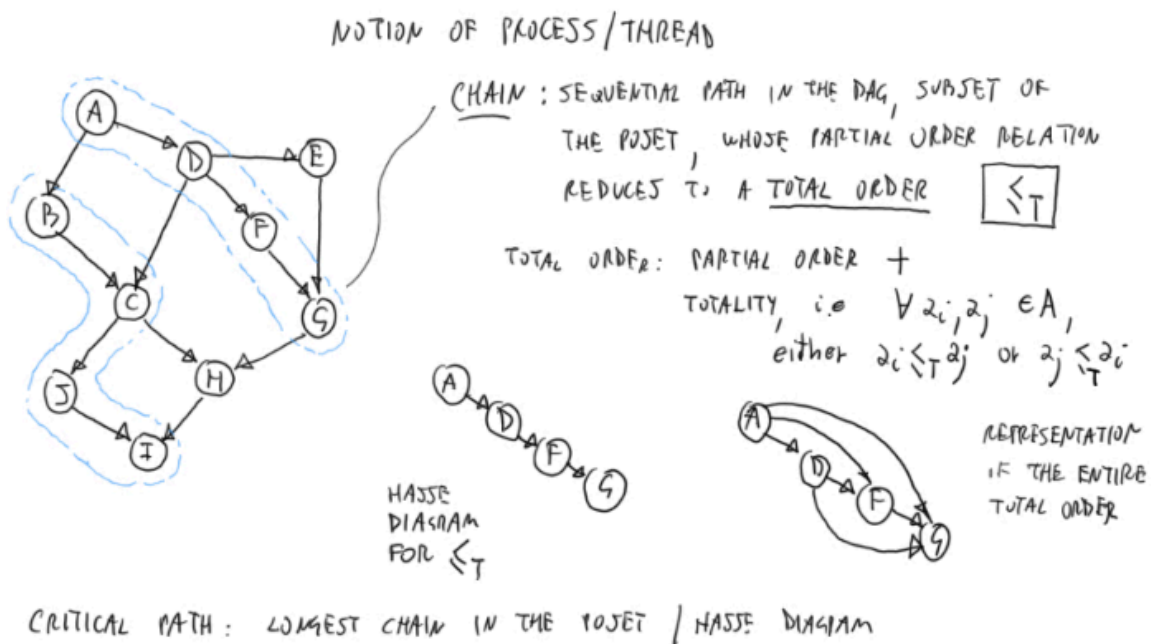
Catena $\rightarrow$ un percorso nel DAG $\rightarrow$ Sottinsieme di elementi del POSET in cui **ogni coppia di elementi è confrontabile**.

In altre parole, dato un sottoinsieme $C \subseteq P$ esso è una catena se $\forall x, y \in C, x \leq y \vee y \leq x$.

Il partial order di C si riduce sempre ad un **total order** ($\leq_T$).

Perché le catene sono importanti?

L'esecuzione di processo/thread seguirà sempre una catena.



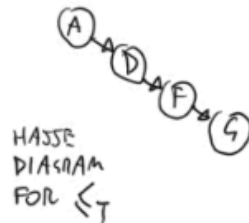
Il percorso $A \rightarrow D \rightarrow F \rightarrow G$ è la sequenza di azioni per arrivare a G partendo da A .

Questo percorso rappresenta l'esecuzione di un certo worker.

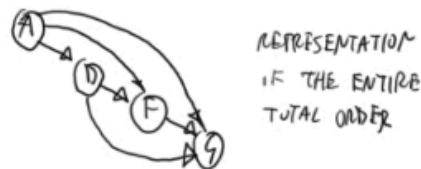
Dal DAG vediamo che E è necessaria per arrivare a G : ma non sappiamo quale worker la eseguirà, ma fino a che E non sarà completata, il nostro worker non potrà eseguire G .

Quindi ci sarà per forza un altro processo/thread che invece eseguirà l'azione E .

Possiamo definire un diagramma di Hasse anche sulle singole catene:



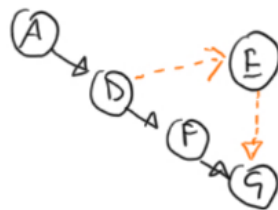
Mentre la rappresentazione del total order è la seguente (include gli archi da una sorgente ad una destinazione raggiungibile da quella sorgente non solo in modo diretto).



La catena più lunga del DAG è detta **percorso critico**

Esecuzione del DAG

L'esecuzione dell'intero DAG può essere portata avanti usando possibilmente molteplici processi.



- Per eseguire G dobbiamo prima eseguire E .
- Per eseguire E dobbiamo prima eseguire D .
- E è esterna alla catena ed è concorrente ad F .

Per eseguire E abbiamo bisogno di un altro processo/thread che la esegua e si sincronizzi con il primo per comunicare quando può eseguire G .

In questo caso, E ed F non sono confrontabili.

Sincronizzazione → meccanismo che forza un vincolo di precedenza a due o più elementi che non appartengono alla stessa catena, che quindi sono indipendenti.

Dentro la catena (*dentro al singolo processo*) non è un problema → attività naturalmente sequenziali.

Sequenzializzare (Topological Sorting): inserire E all'interno della catena, ovvero posizionare tutte le attività in un total order.

$A \rightarrow D \rightarrow E \rightarrow F \rightarrow G$ (E ed F indipendenti → interscambiabili).

Il total order rispetta il relativo partial order? Certo, se $a_i \leq_P a_j \Rightarrow a_j \leq_T a_i$

Il topological sorting viene risolto tramite l'**algoritmo di Kahn**.

Algoritmo di Kahn

Dato un DAG, dobbiamo riorganizzare le azioni in un percorso sequenziale (ovvero in una singola catena) che soddisfi comunque i vincoli di precedenza.

Dividiamo i nodi in tre sottoinsiemi:

1. **SN - Start Nodes**: L'insieme di tutti i nodi senza archi entranti.
2. **S - Solution List**: Inizialmente vuota, verrà riempita dall'algoritmo un nodo alla volta.
3. **M - Middle**: Tutti gli altri nodi.

Step dopo step, sposto i nodi tra i diversi set popolandolo S . Alla fine avrò tutti i nodi nel sottoinsieme S lasciando SN e M vuoti.

```
While  $SN \neq \emptyset$ 
  Pick -  $n \in SN$ 
  Append →  $S.append(n)$ 
   $\forall m \in M$  raggiungibile da  $n$  tramite un arco  $r$ :
    Remove →  $r$  dal grafo
    IF  $m$  non ha altri archi entranti:
      Append →  $SN.append(m)$ 
IF -  $M \neq \emptyset$ :
  Non era un DAG → Errore
ELSE:
  return  $S$ 
```

Legge di Amdahl

La legge di Amdahl da un **limite superiore allo speed-up**, ovvero a quanto le prestazioni di un programma possono migliorare all'aumentare del numero di unità computazionali.

Siano P il **programma originale** nella forma sequenziale e P' , la **versione parallelizzata**.

Sia $T(P)$ il tempo di esecuzione di P , $T(P')$ il tempo di esecuzione di P' (a fronte dello stesso input).

Speed-up $\rightarrow \sigma = \frac{T(P)}{T(P')}$ Indicazione **generica** sull'aumento di performance.

Il programma non è detto che sia interamente parallelizzabile, ci sarà una parte parallelizzabile e una no.

Definiamo $T(P) = s + p$ con:

- s il tempo per svolgere la parte puramente sequenziale.
- p il tempo per eseguire la parte parallelizzabile.

Supponiamo di avere n unità computazionali.

- s non cambia al valore di n .
- p può essere ridotto in $\frac{p}{n}$, parallelizziamo dividendola in n unità.

Sequential Fraction $\rightarrow \alpha = \frac{s}{s+p}$ quanto conviene parallelizzare il programma.

- Se $\alpha = 0 \rightarrow$ Il programma è interamente parallelizzabile e conviene molto farlo.
- Se $\alpha = 1 \rightarrow$ Il programma **non** è parallelizzabile.

Più α è grande e meno è importante la parte parallelizzabile e quindi **meno** vantaggi ottengo dalla parallelizzazione.

NB: le unità computazionali richiedono sincronizzazione, gestione e scambio di informazioni, che comporta un certo overhead computazionale che nella versione sequenziale non era presente. La formula seguente si ottiene dividendo sopra e sotto per $s+p$.

$$\text{Speed-up} \rightarrow \sigma(n, \alpha) = \frac{s+p}{s+\frac{p}{n}} = \frac{1}{\alpha + \frac{1-\alpha}{n}}$$

Da questo otteniamo la **legge di Amdahl**, che da un **limite superiore allo speed-up**.

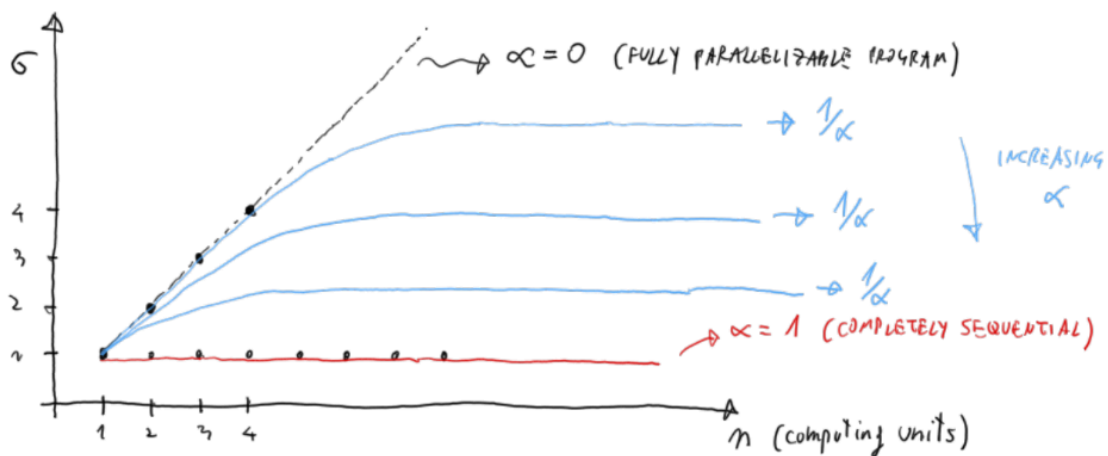
$$\lim_{n \rightarrow \infty} \sigma(n, \alpha) = \frac{s+p}{s} = \frac{1}{\alpha}$$

Grafico al variare di n

Vediamo come varia σ al **variare di n mantenendo α fisso**.

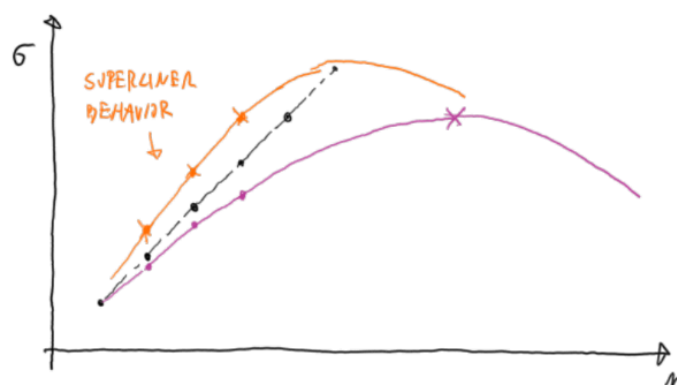
Segue una curva logaritmica.

- Se $\alpha = 1 \rightarrow$ Linea piatta.
- Se $\alpha = 0 \rightarrow$ Retta a 45°.
- Al diminuire di α e allo aumentare di $n \rightarrow$ I benefici aumentano sempre di più.
- All'aumentare di α e allo aumentare di $n \rightarrow$ I benefici diminuiscono sempre di più.



L'aumento è fortemente logaritmico $\rightarrow$ overhead introdotto dalla parallelizzazione (*gestione dei thread*) e all'impatto della parte non parallelizzabile.

Casi in cui all'aumentare di n la curva diminuisce?



In alcuni casi la **linea** è sopra la retta, poi decade $\rightarrow$ **comportamento superlineare**.

Perché decade? Il costo relativo alla gestione dei thread impatta all'aumentare di n .

Perché decade? sistemi molto complessi $\rightarrow$ cache e altre soluzioni hardware o software possono influire molto sul comportamento.

CAPITOLO 5 - JAVA

JVM

JVM → **stack-based machine** → utilizza uno **stack** come struttura dati principale per eseguire operazioni e gestire i dati durante l'esecuzione.

La maggior parte delle istruzioni della JVM lavora direttamente con lo stack invece che con i registri hardware della macchina.

In Java è stato deciso di utilizzare una **stack machine** perché:

- Programmi più compatti (*non richiede di specificare i registri nelle istruzioni*)
- Le stack-based machine sono più **portabili** e semplici **da implementare**.

Processi e thread in Java

Quando esegui un applicazione Java (`java NomeClasse`), il sistema operativo avvia un “**processo pesante**” che esegue la **Java Virtual Machine (JVM)**.

All'interno della JVM vengono eseguiti **uno o più thread**, tra cui il thread `main`, quello relativo al nostro programma

Quindi l'intero programma Java viene eseguito all'interno di un singolo processo pesante (JVM), e le unità di esecuzione interne sono dei thread, figli del processo della JVM.

Esecuzione di un programma Java

Compilazione → Bytecode → Codice Macchina → Esecuzione

Compilazione: Il codice Java viene tradotto in bytecode dal compilatore `javac`.

Bytecode: Formato intermedio di istruzioni eseguibili dalla JVM.

Codice Macchina: JVM traduce il bytecode in codice macchina per l'architettura sottostante (OS + Hardware).

Thread in Java

Definire un thread significa:

- Costruire tutte le strutture dati di supporto, eventualmente specificando attraverso di esse le caratteristiche del thread.
- Fornire le relative azioni di gestione.

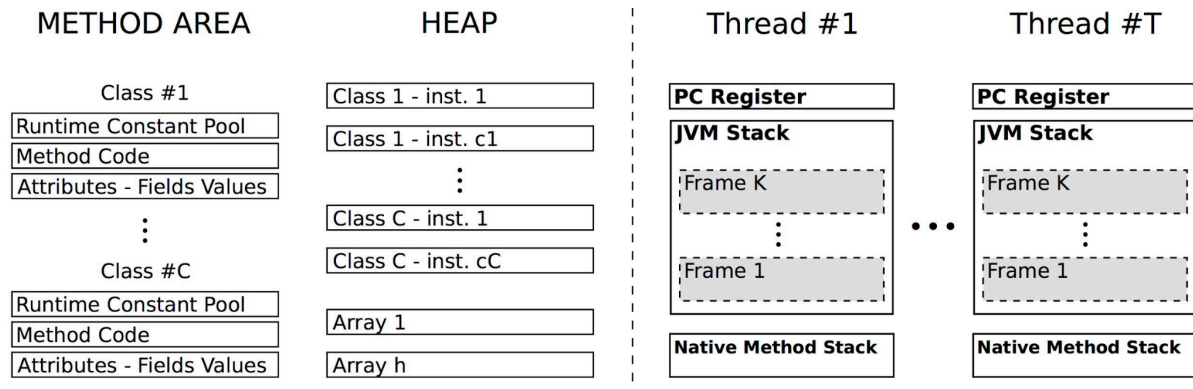
La classe **Thread** in Java rappresenta un'unità di esecuzione indipendente all'interno di un programma, ovvero un **thread di esecuzione**.

Com'è organizzata la memoria della JVM (*Java Runtime Data Area*)?

Oltre ad eseguire i programmi, tra i compiti della JVM c'è la gestione della memoria:

- **Allocazione dinamica** quella fatta con l'operatore **new**.
- **Garbage collection**.

JVM Runtime Data Area



- PC register (*uno per ogni thread*).
- Native Method Stack (*uno per ogni thread*).
- JVM Stack (*uno per ogni thread*).
- Method Area (*uno per tutti i thread*).
- Heap (*uno per tutti i thread*).

PC Register

Contiene l'indirizzo dell'istruzione bytecode corrente o della prossima istruzione da eseguire.

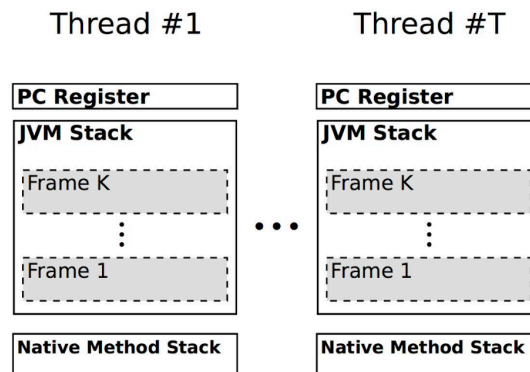
Native Method Stack

Serve per supportare l'esecuzione di metodi nativi, ovvero quei metodi scritti in linguaggi diversi da Java, tipicamente in C o C++, e che vengono eseguiti direttamente dal sistema operativo anziché dalla JVM.

Ogni thread che chiama un metodo nativo ha uno stack separato per la gestione di queste chiamate, analogo al **JVM Stack** che viene usato per i metodi Java normali.

JVM Stack

Lo stack (rispetto allo heap) è **veloce** e con **dimensioni limitate**.



Lo stack è un'area di memoria usata per gestire l'esecuzione dei thread, memorizzando variabili locali, parametri di metodo e informazioni sul contesto delle chiamate di funzione.

Le operazioni del bytecode agiscono direttamente sugli operandi presenti nello stack.

Frame

Utilizzata per rappresentare il contesto di **esecuzione** di un **metodo** all'interno dello **stack** di un thread.

Ogni volta che un metodo viene chiamato, viene creato un nuovo frame. Quando il metodo termina, il frame viene rimosso.

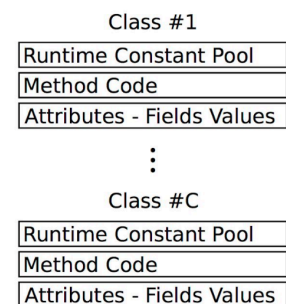
Contesto: Le informazioni necessarie per l'esecuzione di un metodo.

- **Array di variabili locali:** usato per memorizzare i parametri e le variabili locali del metodo.
- **Stack degli operandi:** utilizzato per eseguire operazioni temporanee (es. *somme*, *confronti*, ecc.).
- **Riferimento al runtime constant pool:** questo riferimento permette al frame di accedere alle costanti e ai riferimenti simbolici specifici della classe che contiene il metodo in esecuzione.

Method Area

Contiene il bytecode delle **classi** caricate durante l'esecuzione di un **programma**. Per ogni classe è memorizzato:

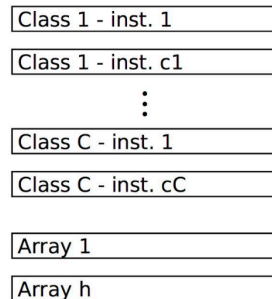
- **Bytecode** (metodi compilati).
- **Metadati** (nome, tipo di variabili,...).
- **Costanti** (costanti statiche finali).
- Il **Runtime Constant Pool**, che agisce in modo simile a una normale tabella dei simboli;



Heap

Utilizzato per la **memorizzazione dinamica delle istanze degli oggetti**.

Contiene gli oggetti (classi e array) con l'operatore **new**, durante l'esecuzione del programma.



STRING CONSTANT POOL

Zona di **memoria ottimizzata dello heap** per conservare le conservate tutte le stringhe costanti, cioè quelle **dichiarate direttamente** nel codice.

```
String saluto = "ciao";
```

Questa zona permette che non vengano duplicate → risparmiando memoria.

Quando si "interna" una stringa, la JVM verifica se una stringa identica è già presente nel **String Pool**.

Se sì, restituisce un riferimento alla stringa esistente. Altrimenti, aggiunge la nuova stringa al pool.

Cosa accade quando un thread chiama una funzione/metodo?

1) Viene eseguito un programma `main()`

1. JVM avviata → viene creata la **Method Area**, dove vengono caricate le informazioni sulle classi, metodi e variabili statiche.
2. Viene creato il **Thread principale (main thread)** → esegue il metodo `main()`.
3. Per il thread main, viene allocato un **Java Stack** → contiene i frame dei metodi invocati.
4. Se il metodo `main()` crea oggetti → Allocati nello Heap.

2) Il main thread crea un nuovo thread

5. Allocato un nuovo **Java Stack** per il thread, separato dallo stack del thread main.

Il nuovo thread è gestito dal **Thread Scheduler** della JVM e può eseguire codice indipendentemente dal thread `main`.

3) Il thread crea un istanza e chiama un metodo della classe `Cane`

6. Un nuovo **stack frame** viene creato nello stack del thread.

```
Cane mioCane = new Cane();  
mioCane.wolf();
```

7. Il thread esegue `new Cane()`.
 - a. Se `Cane` non è già stata caricata → **Class Loader** carica il codice della classe nella **Method Area**.
 - b. JVM alloca memoria nello **Heap** per l'oggetto `mioCane` (*viene chiamato il costruttore*).
 - c. Riferimento all'oggetto → memorizzato nello stack del thread.
8. Il thread chiama i metodi dell'oggetto, creando **frame** per ogni metodo chiamato.

4) Il metodo termina

9. Frame del metodo rimosso dallo stack.
10. Se il metodo ha creato oggetti → restano nello Heap → Garbage Collection.

5) Il thread termina

11. Tutti gli stack frame di quel thread vengono rimossi.
12. Stack deallocato.
13. Se il metodo ha creato riferimenti a oggetti condivisi da altri thread → restano nello Heap → Garbage Collection.

6) Il programma `main` termina

14. Il thread `main` finisce l'esecuzione e il suo stack viene deallocato.
15. Se non ci sono più thread attivi (eccetto i *daemon thread*), la JVM termina.
16. Tutti gli oggetti non più referenziati vengono eliminati dal Garbage Collector.

Quali sono le problematiche associate in questo contesto?

La gestione dei thread (*creazione, distruzione, ecc*) e della garbage collection.

GC - Garbage collection

Responsabile della gestione dello heap → Individua gli oggetti non più referenziati e li rimuove.

Nelle moderne implementazioni JVM, l'heap è partizionato in sezioni gestite con policy diverse, rispetto alla garbage collection.

Gestione dei thread

I problemi di performance nei programmi multithread derivano principalmente dal sovraccarico associato alla gestione dei thread. Questo sovraccarico può manifestarsi in diversi modi:

1. **Context Switch:** il passaggio del controllo da un thread all'altro richiede tempo e comporta lo svuotamento delle cache della CPU (come la cache L1/L2), riducendo l'efficienza e rallentando l'esecuzione successiva.
2. **Contesa sui Lock (Lock Contention):** quando più thread competono per accedere a risorse condivise, si verifica un blocco che forza la serializzazione delle operazioni, riducendo il parallelismo e limitando la scalabilità. È quindi fondamentale minimizzare il tempo di attesa dei thread.
3. **Sincronizzazione della Memoria:** più thread possono avere la necessità di accedere agli ultimi aggiornamenti di un oggetto. La sincronizzazione garantisce la visibilità corretta dei dati, ma ha un costo in termini di performance.

Per ottimizzare le prestazioni, la JVM può applicare tecniche come:

- **Elisione dei Lock (Lock Elision):** rimozione automatica di sincronizzazioni non necessarie per ridurre il sovraccarico.
- **Escape Analysis:** analisi statica che consente di determinare se un oggetto è confinato a un singolo thread. Se sì, le sincronizzazioni associate possono essere eliminate poiché non vi è rischio di concorrenza.

Ad esempio, se abbiamo un codice come sotto in cui c'è un blocco `synchronized()` che rappresenta il meccanismo in Java per rappresentare le `lock`, ovvero un oggetto che agisce come `lock` può essere acceduto da un singolo thread alla volta.

```
synchronized (new Object()) { foo(); }
```

Thread Pooling

Obiettivo → minimizzare il tempo per creare thread.

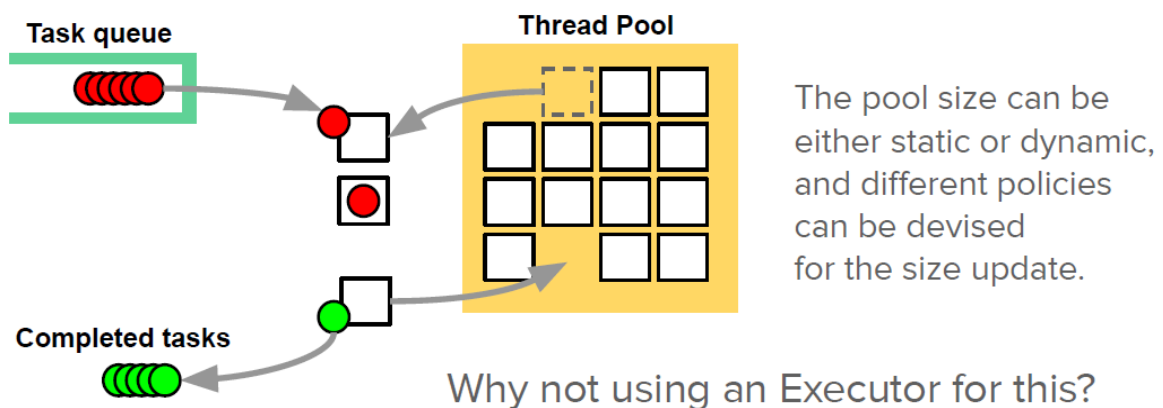
Inoltre bisogna stare attenti al numero di thread presenti nel sistema in relazione al numero di core: se un sistema ha 16 core non ha senso creare 100 thread perchè dovendo essere schedulati su 16 core verrebbe introdotto troppo overhead.

La creazione/eliminazione di un thread determina un sovraccarico computazionale, quindi l'idea è quella di non distruggere mai un thread e quindi di non doverlo mai creare di nuovo.

Supponiamo che vengano creati in anticipo una buona quantità di Thread.

Questa quantità è detta **dimensione del pool** e in certe situazioni deve essere statica (*costante*).

1. Ogni volta che c'è un nuovo task, esso viene inserito in una coda.
2. Viene preso un thread dal pool per eseguire uno dei task nella coda.
3. Una volta completato, il thread ritorna disponibile nel pool.



Il punto quindi è: non distruggere il thread ma renderlo nuovamente disponibile per un'altra attività.

Ci sono “prodotti” già pronti per creare un Thread in Java?

La classe `Thread` con l'interfaccia funzionale `Runnable`.

Classe Thread

In **Java** una classe può **estendere** (ereditare da) **una sola altra classe**. Questo significa che una classe non può avere più di una superclasse diretta.

```
class DatabaseManager {  
    void connect() {  
        System.out.println("Connessione al database...");  
    }  
}
```

Ora vogliamo una classe che:

1. Esegue un thread per aggiornare il database
2. Eredita le funzionalità di `DatabaseManager`

```
class MyThread extends Thread, DatabaseManager { // ✗ ERRORE! Java non lo permette  
    @Override  
    public void run() {  
        System.out.println("Aggiornamento database in corso...");  
    }  
}
```

Se un thread fosse creato estendendo direttamente la classe `Thread`, non potremmo estendere (`extends`) la classe `DatabaseManager`.

Recap - Interfacce e classi astratte

Interfaccia funzionale → contiene **un unico** metodo astratto.

Metodo astratto → dichiarato senza un'implementazione → fornita con `@Override`.

Classe astratta → dichiarata con la keyword `abstract` e non può essere istanziata direttamente.

Interfaccia → sempre astratta, poiché non può avere stato e i suoi metodi sono sempre astratti (*non hanno implementazione*).

Interfaccia Runnable

Invece che estendere `Thread` → **implementare** l'interfaccia funzionale `Runnable`.

`Runnable` contiene il metodo astratto `run()` → codice eseguito nel thread.

```
@FunctionalInterface
public interface Runnable {
    void run();
}
```

```
class DatabaseManager {
    void connect() {
        System.out.println("Connessione al database...");
    }
}

class MyRunnable extends DatabaseManager implements Runnable { // ✓ Ora possiamo estendere
    @Override
    public void run() {
        System.out.println("Aggiornamento database in corso...");
    }

    public static void main(String[] args) {
        MyRunnable task = new MyRunnable();
        Thread thread = new Thread(task);
        thread.start();
    }
}
```

`Runnable` è un'interfaccia funzionale → espressioni Lambda per definire il codice Thread → non abbiamo bisogno di una classe separata.

```
public class Main {
    public static void main(String[] args) {
        Thread thread = new Thread(() -> System.out.println(Thread.currentThread().getName()))
        thread.start();
    }
}
```

♦ `thread.start()` VS `thread.run()`

- `start()` : Crea un nuovo thread e invoca `run()`, eseguendo il codice in parallelo rispetto al thread principale.
- `run()` chiamato direttamente non crea un nuovo thread, ma esegue il codice nel thread principale (senza concorrenza).

Spiegare Executor (thread pooling), come funziona?

L'interfaccia `Executor` fa parte del pacchetto `java.util.concurrent` e rappresenta un'interfaccia fondamentale per la gestione di thread e attività asincrone.

Fornisce un'astrazione per eseguire task senza doversi occupare direttamente della creazione e gestione dei thread.

L'utilizzo è molto semplice:

1. Definiamo `task` usando `Runnable` o `Callable`.
2. Diamo `task` allo `Executor`.
3. `Executor` lo eseguirà occupandosi lui della gestione.

```
public class ExecutorExample {
    public static void main(String[] args) {
        Executor ex = Executors.newSingleThreadExecutor(); // Definisce lo Executor
        Runnable task = () -> System.out.println("ciao"); // Definisce il task.
        ex.execute(task); // Esegue il task.
    }
}
```

Implementazione **personalizzata** di `Executor`:

```
public class CustomExecutor implements Executor {
    @Override
    public void execute(Runnable myTask) {
        new Thread(myTask).start(); // Esegue ogni task in un nuovo thread
    }

    public static void main(String[] args) {
        Executor customExecutor = new CustomExecutor();
        // Invia due task all'executor
        customExecutor.execute(() -> System.out.println("ciao1"));
        customExecutor.execute(() -> System.out.println("ciao2"));
    }
}
```

Implementazioni personalizzate di `execute()`.

```
@Override
public void execute(Runnable myTask) {
    myTask.run(); // Semplice chiamata di funzione, senza creare un thread.
    // La funzione è eseguita dal thread principale.
}

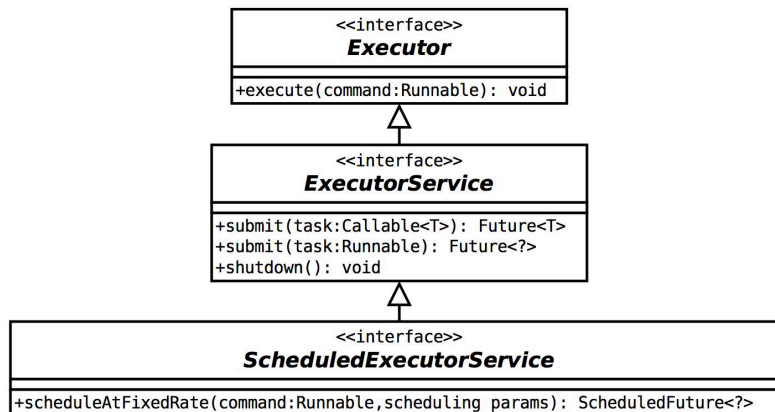
@Override
public void execute(Runnable myTask) {
    // viene creato un thread il cui body corrisponde a myTask.
    new Thread(myTask).start(); // Viene mandato in esecuzione.
}
```

Vantaggi dell'interfaccia `Executor`

- **Separazione delle preoccupazioni:** si separa la logica di invio dei task dall'implementazione della gestione dei thread.

- **Astrazione:** `Executor` astrae la gestione dei thread, evitando di crearli manualmente.
- **Gestione scalabile dei thread:** `Executor` usa thread pooling per gestire efficientemente l'esecuzione dei `Runnable`.
- **Codice più leggibile e flessibile:** il framework consente di astrarre l'uso di thread in modo uniforme e gestibile.

Executor - le varie classi, parametri ecc.



1 Executor (Base)

- Definisce il metodo `execute(Runnable command)`, che accetta un `Runnable` e lo esegue.
- È l'interfaccia più semplice per la gestione di task concorrenti.

2 ExecutorService (Più avanzato)

- Estende `Executor` e aggiunge metodi più sofisticati:
 - `submit(Callable<T>)`: Esegue un task e restituisce un `Future<T>` per ottenere il risultato.
 - `submit(Runnable)`: Esegue un task e restituisce un `Future<?>`, che permette di controllarne l'esecuzione.
 - `shutdown()`: Ferma l'esecuzione dei task.
- Permette di gestire i thread in modo più controllato rispetto a `Executor`.

3 ScheduledExecutorService (Task periodici)

- Estende `ExecutorService` e permette di schedare task ricorrenti.
- Il metodo `scheduleAtFixedRate(Runnable, scheduling params)` permette di eseguire periodicamente un task con una frequenza fissa.

A cosa serve l'interfaccia funzionale Callable?

`Callable<T>` è usata per definire **task concorrenti che restituiscono un valore e possono lanciare eccezioni**.

```
public interface Callable<T> {  
    T call() throws Exception;  
}  
  
Callable<Integer> task = () -> {  
    Thread.sleep(1000); return 42;  
};
```

Usata con gli `Executor` per eseguire operazioni asincrone in thread pool, fornendo un risultato tramite un oggetto `Future<V>`.

- ♦ Differenza con `Runnable`?
 - `Runnable` → Non restituisce nulla (`void`) e non lancia eccezioni.
 - `Callable<V>` → Restituisce un valore (`V`), può lanciare eccezioni (`Exception`).

ExecutorService

`ExecutorService` fornisce un meccanismo per **separare l'invio di un task dalla sua esecuzione effettiva**.

Il vantaggio è quello di poter far lavorare dei thread in background (*grazie al thread pooling*) e recuperare i risultati solo quando sono strettamente necessari.

1. Un task implementa `Callable<V>`, definendo `call()`, che restituisce un valore.
2. `ExecutorService` esegue il task usando `submit()`.
3. Restituisce un `Future<V>`.
4. Valore recuperato con `future.get()`.

Perché `ExecutorService` è utile?

La gestione delle esecuzioni è complessa: devono essere considerati alcuni aspetti:

Non sappiamo quando il task restituirà il risultato, e quando potrebbe servirci potrebbe essere troppo presto → inconsistenza.

Serve **sincronizzazione** per ottenere il risultato in modo sicuro.

Cosa sono i Java Futures?

Quando va usata `Callable`? → Quando un task deve **restituire un valore**.

`Runnable` non ha un valore di ritorno.

`Callable` è un'interfaccia funzionale con un metodo `call()` che restituisce un `Future`.

`Future` è un oggetto **thread-safe** utilizzato per rappresentare il **risultato di un'operazione asincrona**, che potrebbe non essere ancora completata.

```
public interface Future<V> {  
    boolean isDone();  
    V get(); //also a timed version exists  
    boolean cancel(boolean m);  
    boolean isCancelled();  
}
```

 **Concetto chiave:** `Future` è un proxy per un valore che sarà disponibile in futuro.

A che cosa servono i Java Futures e quali sono i vantaggi che portano in un contesto concorrente?

Se un `Callable<T> task` viene inviato a un `ExecutorService` con il metodo `submit()`, esso restituisce un `Future<T>` che permette di:

- **Ottenere il valore calcolato** con `get()`, se non è pronto → bloccante.
- **Controllare se il task è terminato** con `isDone()`.
- **Annullare il task** con `cancel()`.

Quale è la differenza tra un Promise e un Future?

Future → L'oggetto che **legge il risultato** se disponibile.

La promise è la funzione incaricata di impostare il valore ad un `Future`.

Promise → L'entità che **imposta il risultato** di un'operazione asincrona.

```

public class FutureExample {
    public static void main(String[] args) {
        // Creazione di un ExecutorService con un thread pool di 2 thread.
        ExecutorService executor = Executors.newFixedThreadPool(2);
        // Task che calcola il quadrato di un numero.
        Callable<Integer> task = () -> {
            Thread.sleep(2000); // Simula un'operazione lunga.
            return 42; // Risultato.
        };
        // Sottomette il task all'ExecutorService con submit.
        Future<Integer> future = executor.submit(task);
        // La promise è stata restituita, il calcolo è in esecuzione.
        System.out.println("Task sottomesso. Aspetto il risultato...");
        try {
            Integer result = future.get(); // Recupera il risultato del task
            // Bloccante fino a quando il risultato è disponibile
            System.out.println("Risultato: " + result);
        } catch (Exception e) {
            System.err.println(e.getMessage());
        } finally {
            executor.shutdown(); // Arresta Executor.
        }
    }
}

```

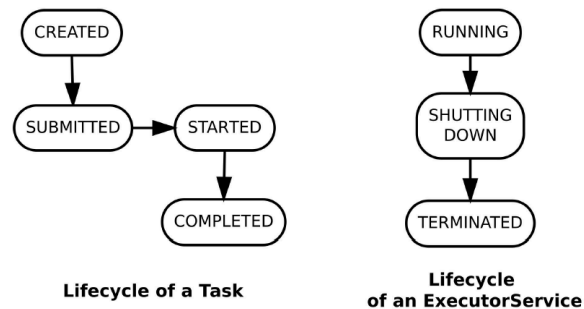
♦ Cosa succede quando chiamiamo `submit()` ?

1. Il task viene inviato a un pool di thread per l'esecuzione.
2. `submit()` restituisce immediatamente un `Future<T>`, che rappresenta il risultato futuro dell'operazione.
3. Il task viene eseguito in background su uno dei thread disponibili del pool.
4. Quando il task termina, il risultato viene scritto nel `Future`.

♦ Quando viene scritto il risultato nel `Future` ?

- Il valore viene calcolato nel momento in cui il thread che esegue il task termina il suo lavoro.
- Può essere recuperato con `future.get()`, che blocca il thread chiamante fino alla disponibilità del risultato.

Come può essere fermato un Executor?



Dipende da cosa vogliamo fare.

In un **Executor** potrebbero esserci molti task nella coda che sono pronti ad essere eseguiti.

Se ci sono dei task pronti nell **Executor** da terminare, prima vengono eseguiti, poi viene terminato.

Queste opzioni si scelgono tramite il relativo metodo di **ExecutorService**:

- **shutdown()**: **Termina in modo ordinato**: non accetta nuovi task, ma completa quelli in esecuzione o in coda.
- **shutdownNow()**: **Forza l'arresto immediato**: tenta di interrompere i thread attivi e restituisce una lista dei task non ancora avviati.

Possiamo controllare lo stato dell **Executor** con i seguenti metodi:

- **isShutdown()** restituisce lo stato dell **Executor**.
- **isTerminated()** controlla se tutte le attività sono state terminate.

I metodi sopra sono non bloccanti; per attendere il completamento di tutte le attività, chiama **awaitTermination()**.

CAPITOLO 6 - Clock per il tempo logico

Definisci cos'è un "evento" in questo contesto

l'esecuzione di un processo → sequenza di eventi.

- **Eventi Interni:** azioni significative, che non coinvolgono altri processi.
- **Eventi di comunicazione:** invio/ricezione messaggi (*non necessariamente asincrono*).

Ogni processo è **sequenziale**: → eventi eseguiti uno dopo l'altro (*tot. ordinati*).

Il k° evento eseguito dal processo p_i è indicato come e_i^k .

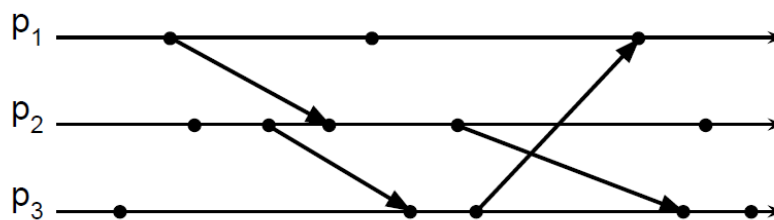
Cronologia (history) La sequenza di tutti gli eventi eseguiti da p_i .

$$h_i = e_i^1, e_i^2, \dots, e_i^k, \dots, e_i^n$$

Prefisso: P è rappresentazione **parziale** (*fino a un certo punto*) della sequenza degli eventi che costituiscono l'evoluzione di p_i .

$P = e_i^1, e_i^2, \dots, e_i^k \rightarrow$ prefisso fino al k° evento.

Descrivi un diagramma spazio-tempo



Per ogni messaggio gestito in una comunicazione asincrona, possiamo definire un ordinamento degli eventi correlati:

- **Punti** → eventi interni (*senza freccia*).
- **Punti con freccia** → Coppia di eventi di comunicazione `send(m)` e `receive(m)`.

Consistent Cut

Consistent Cut → Prefisso che rappresenta uno **stato globale valido** del sistema.

Stato Globale Valido → Se rispetta la causalità degli eventi, ovvero:

Se un evento e è incluso nel cut → tutti gli eventi che lo **precedono causalmente** (*da cui e dipende*) sono inclusi nel cut.

Se $e \xrightarrow{HB} f$, ed f è nel cut → anche e è nel cut.

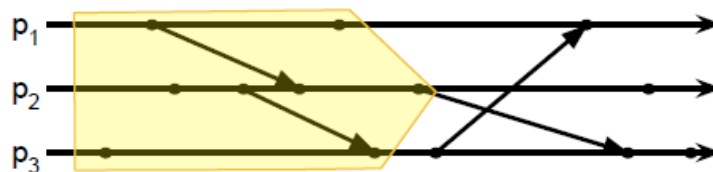
Come si verifica questa cosa?

Non si può includere un evento nel cut se non si sono già inclusi gli eventi da cui dipende causalmente (*ovvero che lo hanno causato*).

Cosa permette di fare il consistent cut? → Stabilire un punto di progressione.

Punto di progressione → Invece di elencare tutti gli eventi del prefisso, possiamo rappresentare un prefisso solo tramite la frontiera del taglio.

Frontiera del taglio → Il set di tutti gli ultimi eventi del cut (sotto, i 3 più a destra).



Un Cut è **coerente** (consistent) se $\forall$ messaggio m tale che $receive(m)$ sia nel cut, anche la relativa $send(m)$ è nel cut.

Relazione HB - Happens-Before

Relazione tra due eventi che tiene conto di tutte le precedenze durante l'esecuzione: $\xrightarrow{HB}$.

HB0 (transitività)

Siano gli eventi e, f, g e supponiamo che $e \xrightarrow{HB} f \wedge f \xrightarrow{HB} g$.

Allora $e \xrightarrow{HB} g$.

Ovvero se e accade prima di f e f accade prima di g , allora e accade prima di g .

HB1 (ordinamento in corso)

Siano gli eventi e, f nello stesso processo p_i e che $e \xrightarrow{i} f$.

Allora $e \xrightarrow{HB} f$.

Se nel processo p_i , l'evento e accade prima di f , seguendo l'ordine di sequenzialità del processo, allora c'è una relazione di HB di e che viene prima di f .

HB2 (comunicazione asincrona)

Per qualsiasi evento $e = \text{send}(m)$ (non bloccante).

Sia l'evento corrispondente $f = \text{receive}(m)$ per lo stesso m , vale $e \xrightarrow{HB} f$.

Non posso ricevere qualcosa che non è stato ancora inviato!

Eventi concorrenti

Due eventi e, f sono detti **concorrenti** ($e || f$) se e solo se $\neg(e \xrightarrow{HB} f) \wedge \neg(f \xrightarrow{HB} e)$.

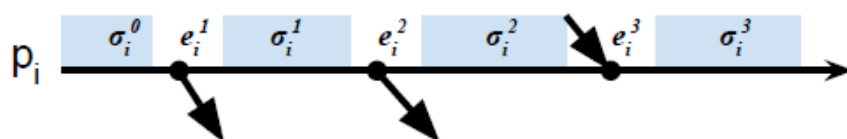
Ovvero se e solo se non esiste una relazione HB tra di essi.

Concorrenti $\rightarrow$ I due eventi non sono confrontabili (indipendenti) e quindi possono avvenire anche contemporaneamente.

Eventi e stati locali

Calcolo locale $\rightarrow$ sequenza di stati locali, con transizioni innescate da eventi.

Nella figura sotto abbiamo una serie di eventi nel tempo, un evento causa il cambiamento dello stato interno di un processo.



In **azzurro** in figura sono indicati gli **stati locali**.

Quando avviene l'evento e_i^1 , il processo andrà nello stato locale σ_i^1 .

Gli eventi sono ordinati logicamente, cioè ogni evento successivo avviene dopo un evento precedente.

Stato Globale

Stato Globale → Configurazione del sistema in un dato momento.

Configurazione del sistema → Una combinazione degli stati locali di tutti i processi del sistema.

Uno "*stato globale*" è una raccolta di uno stato locale per processo: $\Sigma = [\sigma_i, \sigma_j, \dots, \sigma_n]$

Ogni stato globale del sistema rappresenta un sottoinsieme di eventi accaduti.

Sistema Distribuito → Non esiste un clock globale sincronizzato.

Non esiste un clock globale sincronizzato → Tempo globale non esiste.

Tempo globale non esiste (i sistemi non sono sincronizzati) → Alcuni stati globali non confrontabili.

- Alcuni stati globali sono confrontabili, ad esempio se uno deriva da un altro tramite eventi ordinati.
- Altri stati globali sono **indipendenti** se provengono da eventi che non sono confrontabili (es. due processi diversi che eseguono operazioni in parallelo senza comunicare).

Di conseguenza, l'insieme degli stati globali è strutturato secondo un **ordine parziale**, dove alcuni stati possono essere confrontati (cioè uno deriva dall'altro), mentre altri restano indipendenti.

Alcuni stati globali non confrontabili → L'insieme degli stati globali è un partial order.

Spiega il concetto di Clock logico

Se dobbiamo occuparci degli ordinamenti relativi delle occorrenze degli eventi, possiamo sostituire il "*tempo globale*" con un **indice** che fa riferimento semplicemente all'ordine degli eventi.

Ma a noi interessa solo l'**ordine in cui** collocare un evento rispetto ad altri eventi.

Clock logico $C(\bullet) \rightarrow$ mezzo per mappare gli eventi su un partial order, ovvero su un ordine che rispetta la causalità degli eventi.

Il clock logico **che vogliamo** deve soddisfare le seguenti proprietà:

- **Consistenza:** Se $e \xrightarrow{HB} f \Rightarrow C(e) < C(f)$.
- **Dipendenza Causale:** Se $C(e) < C(f) \Rightarrow e \xrightarrow{HB} f$.

Nel nostro caso $C(\bullet)$ è un **numero intero** e quindi può essere detto "*tempo lineare*".

Cosa sono i Lamport Timestamps?

Il clock logico di Lamport T_{lp} di un evento e è associato alla **lunghezza della catena HB** più lunga per raggiungere e .

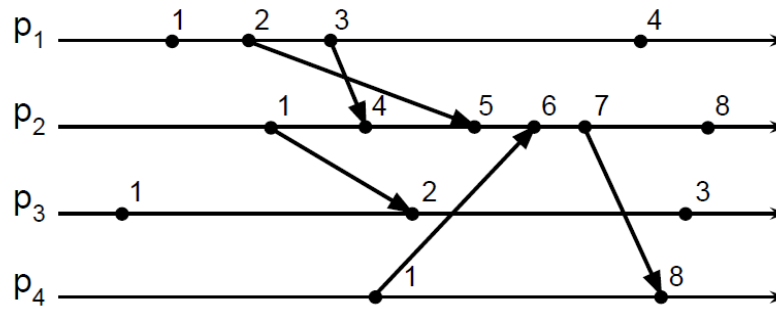
Il loro scopo è quello di dare un ordinamento agli eventi in un sistema distribuito.

Ogni processo p_i mantiene un contatore locale $clock_i$ (un intero) che viene aumentato ogni volta che avviene un evento secondo le seguenti regole:

- Inizialmente $clock_i = 0$.
- Per tutti gli eventi **tranne** $receive(m)$:
 - $clock_i \leftarrow clock_i + 1$
 - $T_{lp}(e) \leftarrow clock_i$

Nelle $send(m)$ viene trasportato anche il timestamp $t_s(m)$ del mittente.

- Per le $receive(m)$ il processo ha ricevuto da un altro processo e quindi deve verificare **quale è il timestamp più aggiornato**.
 - $t_s(m) \leftarrow clock_j$
 - $clock_i \leftarrow \max(clock_i, t_s(m)) + 1$
 - $T_{lp}(e) \leftarrow clock_i$



Vediamo che p_1 (arrivato al clock 2 e al clock 3) invia un messaggio a p_2 .

Ma il messaggio inviato al clock 3, per una strana ragione, arriva prima di quello inviato al clock 2.

Seguendo p_2 da 1 passa subito a 4.

Questo perché in entrambe le $send(m)$ fatte in p_1 si è trasportato il timestamp del momento in cui è stato inviato il messaggio e quindi p_2 si è adattato (caso 3). Poiché:

$$clock_2 = 1 < t_{s1}(m) = 3 \Rightarrow clock_2 \leftarrow t_{s1}(m) + 1 = 4$$

Proprietà del Lamport timestamp

Consistenza del Clock (coerenza): Se $e \xrightarrow{HB} f \Rightarrow T_{Lp}(e) < T_{Lp}(f)$ garantito dalla costruzione stessa dell'algoritmo di Lamport.

Per contrapposizione, se $T_{Lp}(e) \leq T_{Lp}(f) \Rightarrow \neg(f \xrightarrow{HB} e)$.

- Se $T_{Lp}(e) \leq T_{Lp}(f) \rightarrow e$ è accaduto prima di f , oppure $e || f$.
- Se due eventi hanno lo stesso timestamp di Lamport, significa che sono **indipendenti** (concorrenti).

Nessuno dei due può aver causato l'altro, perché i timestamp di Lamport rispettano solo la relazione **HB**, ma non distinguono eventi indipendenti: $T_{Lp}(e) = T_{Lp}(f) \Rightarrow e || f$.

Il timestamp di Lamport non soddisfa il requisito di dipendenza causale e quindi non possiamo essere certi della causalità di due eventi.

Non soddisfa il requisito $\rightarrow$ può accadere che $T_{Lp}(e) < T_{Lp}(f) \wedge \neg(e \xrightarrow{HB} f)$.

Può accadere che e risulti precedente a f senza una relazione **HB** tra i due.

Quindi $T_{Lp}(e) < T_{Lp}(f) \nRightarrow e \xrightarrow{HB} f \rightarrow$ **non dimostra la dipendenza causale.**

Ad esempio, questa cosa accade quando il mittente invia due messaggi che non sono legati in alcun modo tra loro.

```

P1: (e1) --- [Send M1] --- (e2) --- [Send M2] --- (e3)
P2:          (f1) --- [Receive M1] --- (f2) --- [Receive M2] --- (f3)

```

Dove:

- $e1, e2, e3$ sono eventi locali in $P1$.
- $f1, f2, f3$ sono eventi locali in $P2$.
- $M1$ e $M2$ sono messaggi inviati da $P1 \downarrow P2$.
- $TLp(e2) = 5$
- $TLp(e3) = 6$
- $TLp(f2) = 7$
- $TLp(f3) = 8$

Osserviamo che $TLp(e2) < TLp(f3)$, ma **NON** esiste una relazione di causalità tra $e2$ e $f3$!

Perché?

- $e2$ invia $M1$, ma $f3$ è il risultato di un altro messaggio ($M2$), non dipende da $e2$.

- I due eventi sono indipendenti tra loro.

● Errore comune:

Pensare che se $TLp(e) < TLp(f)$, allora e è sicuramente avvenuto "prima" di f nel senso di causalità.

➡ **FALSO**, perché TLp garantisce solo un **ordine totale** e non un ordine causale!

✦ Perché succede questo?

- Lamport Timestamp segue la regola: *se un evento accade prima di un altro nel tempo, deve avere un timestamp minore.*
- Ma non distingue tra eventi causalmente connessi e eventi indipendenti.

Totally Ordered Lamport Timestamp

- ✓ Permette un confronto sempre univoco tra eventi.
 - ✓ Utile per sistemi distribuiti che necessitano di un ordine globale.
 - ✓ Evita situazioni di timestamp uguali in processi diversi.
 - ✗ Non garantisce la dipendenza causale tra eventi.
- 💡 Se vogliamo tracciare la causalità, dobbiamo usare i Vector Clocks!

Obiettivo: Generare un numero intero che rappresenti in modo **totale e univoco** il timestamp di ogni evento.

T_{Lp} (ordine parziale) può essere utilizzato per **ottenere un ordine totale** T_{Lt} su tutti gli eventi, in modo che sia coerente con la relazione HB .

- L'ordine parziale lascia elementi non confrontabili (come eventi indipendenti in un sistema distribuito).
- L'ordine totale forza il confronto tra tutti gli elementi, anche quando non esiste una relazione naturale tra loro.
- Il Totally Ordered Lamport Timestamp (T_{Lt}) è utile per ordinare eventi in modo globale, anche se non garantisce causalità.

Come funzionano?

Per il processo p , sia $T_{Lt} = (T_{Lp}, i)$ il quale afferma che:

$$(a, b) < (c, d) \Leftrightarrow \{ (a < c) \vee [(a = c) \wedge (b < d)] \}$$

- Se $a < c$, l'evento con timestamp a è chiaramente precedente.
- Se $a = c$, confrontiamo il valore di b (indice del processo), così da avere un ordine totale.

Forza il confronto → Non potrà mai verificarsi $T_{Lt,j} = T_{Lt,i}$ poiché gli identificatori (PID) sono per forza tutti diversi tra processi.

$$T_{Lp} \ll B + i,$$

Dove:

- $\ll$ (bit shift a sinistra): moltiplica T_{Lp} per una potenza di 2, "spostando" lo spazio per l'indice del processo.
- $B = \lceil \log_2(n) \rceil$: numero di bit necessari per rappresentare n processi.
- i : identificatore del processo.

Vector timestamps, differenze con i Lamport e proprietà.

Vogliamo un **clock in grado di indicare la dipendenza causale**:

Ovvero, se $C(e) < C(f) \Rightarrow e \xrightarrow{HB} f$

Questo **non è possibile con il Lamport timestamp** poiché è possibile avere

$$T_{Lp}(e) < T_{Lp}(f) \wedge \neg(e \xrightarrow{HB} f).$$

Timestamp vettoriali $\rightarrow$ mantenere, per ogni processo, un **vettore di contatori** V_i di n (con n il numero di processi) interi.

- $V_i[i]$ è il contatore degli eventi all'interno di p_i , ovvero l'informazione più recente riguardo la progressione di p_i .
- $V_i[j]$ è il valore più recente di $V_j[j]$ visto da p_i , ovvero l'informazione più recente riguardo la progressione di p_j .

Ogni volta che p_i invia un messaggio, il valore più recente di $V_i[i]$ viene trasportato nel messaggio e il processo destinatario aggiornerà il relativo contatore.

Algoritmo per timestamp vettoriali

Ad ogni evento e , il contatore vettoriale locale V_i viene aggiornato e il suo valore viene assegnato all'evento come timestamp vettoriale $T_v(e)$.

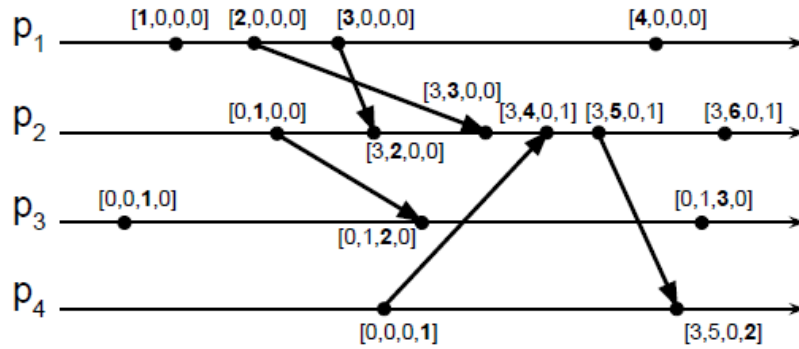
1. Per tutti i processi p_i , ad ogni operazione fatta viene effettuato $V_i[i] \leftarrow V_i[i] + 1$
2. In caso di $e = \text{send}(m)$: $t_s(m) \leftarrow T_v(e)$ (trasportato nel messaggio m).

L'intero vettore $T_v(e)$ viene trasportato in m .

3. In caso di $e = \text{recv}(m)$, **per ogni cella i : $V_i \leftarrow \max\{V_i, t_s(m)[i]\} + 1$**
4. $T_v(e) \leftarrow V_i$ (perché nella receive prende il max quindi deve essere aggiornato).

In questo esempio vediamo che nel primo valore:

- Per p_1 [1,0,0,0].
- Per p_2 [0,1,0,0].
- Per p_3 [0,0,1,0]
- Per p_4 [0,0,0,1].



Osserviamo che p_1 non conosce mai nulla riguardo agli altri processi, perché non riceve mai un messaggio.

Nel secondo punto di p_2 , esso conosce già che p_1 è arrivato nella posizione 3 (*primo elemento della lista*), quindi non viene cambiato nulla.

Come compari due vector timestamp?

Dati due timestamp vettoriali T_1 e T_2 , definiamo:

1. $T_1 \leq T_2 \Leftrightarrow T_1[k] \leq T_2[k] \forall k \in [1, n]$ *confronto elemento per elemento.*
2. $T_1 = T_2 \Leftrightarrow T_1[k] = T_2[k] \forall k \in [1, n]$ *confronto elemento per elemento.*
3. $T_1 < T_2 \Leftrightarrow (T_1 \leq T_2) \wedge (T_1 \neq T_2)$
4. $T_1 \parallel T_2 \Leftrightarrow T_1 \not\leq T_2 \wedge T_2 \not\leq T_1$

“Not Less” : $T_v(e) \not\leq T_v(f) \rightarrow \exists i \in [1, n] : T_v(e)[i] > T_v(f)[i]$

Le ultime due definizioni riguardano la precedenza e la concorrenza *HB*.

Dimostrazione della Strong C.C

I timestamp vettoriali soddisfano entrambi i requisiti.

- **Consistenza del clock:** $e \xrightarrow{HB} f \Rightarrow T_v(e) < T_v(f)$.
- **Consistenza Causale Forte / Dipendenza causale:** $T_v(e) < T_v(f) \Rightarrow e \xrightarrow{HB} f$.

Dimostrazione

Per contrapposizione: dimostriamo che se $\neg(e \xrightarrow{HB} f) \Rightarrow T_v(e) \not< T_v(f)$.

$$T_v(e) \not< T_v(f) \Leftrightarrow \exists i \in [1, n] : T_v(e)[i] > T_v(f)[i].$$

Dobbiamo quindi dimostrare che se $\neg(e \xrightarrow{HB} f)$ allora $\exists i \in [1, n] : T_v(e)[i] > T_v(f)[i]$

1. Sia e un evento in p_i , ed f un evento in p_j .
2. Supp. che $\neg(e \xrightarrow{HB} f)$ sia vera.
3. Supponiamo che prima di e , p_i aumenti $V_i[i]$, quindi $T_v(e)[i] \leftarrow t$.
4. Data (2) non c'è modo per t di propagarsi a p_j prima che f accada.
5. Quindi il valore conservato da p_j al momento di $f \rightarrow T_v(f)[i] < t$.
6. Quindi $T_v(e) \not< T_v(f)$.

Quali sono gli utilizzi dei vector timestamps?

Verificare la dipendenza causale tra due eventi e di **verificare la coerenza di un taglio**.

Sia $F(C) = \{f_1, \dots, f_n\}$ l'insieme di eventi di frontiera di un taglio C .

Sia il **timestamp del taglio** $T_v(C) = \max\{T_v(f_1), \dots, T_v(f_n)\}$

- f_i è l'evento selezionato sul processo p_i .

Il timestamp del taglio sostanzialmente è il vector timestamp maggiore tra i timestamp degli eventi di frontiera.

Il taglio C è coerente se $\forall i, T_v(C)[i] = T_v(f_i)[i]$.

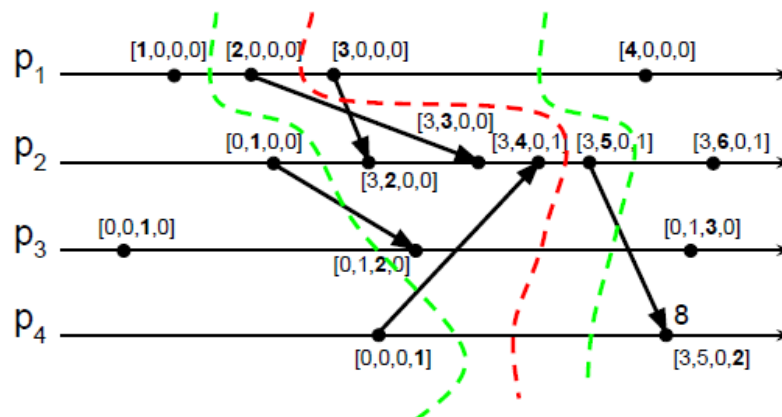
Il valore massimo i^o valore deve sempre essere sul processo p_i .

Se così non fosse, significherebbe che un altro processo diverso da p_i ha un valore $T_v(f_i)[i]$ maggiore di quello detenuto da p_i stesso. E ciò implicherebbe che un altro processo nel taglio conosce il futuro di p_i .

Questo accade quando la send è esclusa dal cut mentre la receive si.

Se controlliamo il taglio **verde**, vediamo che la proprietà è applicata.

Nel taglio **rosso** non è applicata perché p_2 conosce il futuro di p_1 , visto che al suo punto 3, ha valore 3 come primo elemento dell'array, mentre p_1 ha 2 come primo elemento al suo punto 2.



Come si può notare, secondo la definizione data in precedenza di taglio consistente, la send dell'evento 3 del processo 1 non è inclusa nel taglio, mentre è inclusa la receive nel processo due, quindi ad una receive inclusa non corrisponde anche al relativa send, quindi non può essere consistente.

CAPITOLO 8 - COMMUNICATION SUPPORT FOR DISTRIBUTED SYSTEM

Comunicazione diretta: ogni parte coinvolta conosce direttamente l'altra entità con cui sta comunicando.

Comunicazione indiretta: la parte coinvolta non sa chi riceverà un messaggio, poiché usa un intermediario.

Comunicazione diretta

Ci sono tre opzioni fondamentali:

- Protocollo di **richiesta/risposta** (*http*).
- **RPC** - Remote Procedure Call.
- **RMI** - Remote Method Invocation.

Le chiamate locali di solito hanno una semantica di tipo "*exactly-once*" (*l'operazione viene effettuata una sola volta e con successo*), ma essa non è possibile in un ambiente distribuito.

Per le chiamate remote **in presenza di comportamenti errati** possono esserci i comportamenti espressi in tabella.

Filtraggio dei duplicati, ritrasmissione o ri-esecuzione è fatto tutto a livello middleware.

In caso di trasmissione della stessa richiesta per una seconda volta:

1. **Re-execution:** La risposta viene ricalcolata e re-inviata; il server non ignora i duplicati e tratta tutte le richieste come se fossero uniche.
2. **Re-transmission:** Viene riproposta la risposta calcolata prima.

Che cos'è una RPC (Remote Procedure Call)?

Forma di comunicazione **diretta** (a livello di applicazione) **interprocesso** tra un client e un server.

RPC → Esecuzione di una procedura nel server con restituzione del risultato al client.

Nel nostro programma quando vogliamo effettuare una RPC non dobbiamo specificare tutti i dettagli possibili, vogliamo mantenere queste chiamate il più possibile simili a chiamate ordinarie nel codice.

1. Programmazione basata sull'interfaccia → Astrazione, "*portabilità*".
2. È possibile scegliere/implementare diverse semantiche di chiamata.
3. Trasparenza, rispetto all'accesso (*locale vs remoto*), posizione, ecc. (*concorrenza, replica, errore, mobilità, prestazioni, scalabilità*).

Come sono passati i parametri?

Le controparti:

- Non condividono lo spazio di indirizzamento poiché risiedono in macchine diverse → Non si può fare un passaggio per riferimento.
- Non condividono il linguaggio → Marshalling.

Il passaggio dei parametri nelle RPC è sempre per valore.

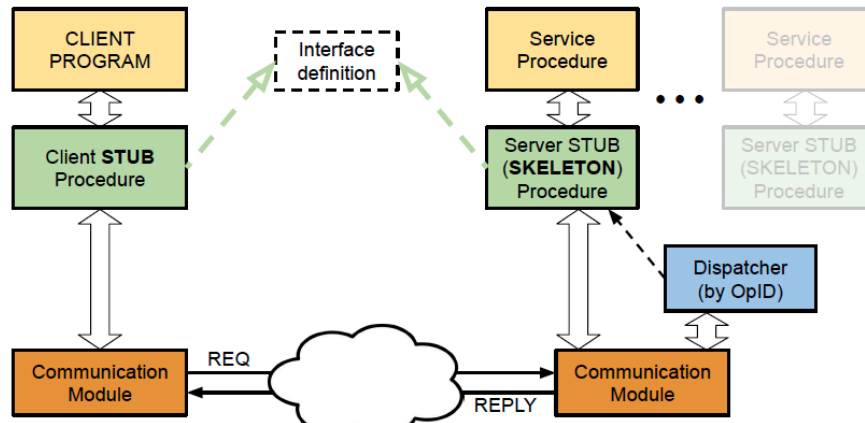
- **IN:** Parametro usato solo per l'input. Il client invia un valore al server, ma il server **non restituisce alcuna modifica** su quel parametro.
- **OUT:** Parametro usato solo per l'output. Il client non invia un valore iniziale, ma si aspetta che il server **riempia questo parametro** e lo restituisca nella risposta.
- **INOUT:** Parametro usato sia per input che per output. Il client invia un valore iniziale, il server può modificarlo e **restituirlo** al client.

Qual è l'architettura del sistema che usa gli RPC?

Si basa su un'architettura di tipo **client-server**.

Client program e **service procedure** possono essere scritti in due linguaggi diversi, quindi serve una **soluzione middleware** che si interpone tra le due parti.

Tutto ciò che non è **client program** e **service procedure** è considerato **middleware**.



Client STUB

Agisce come un *proxy* per il client, nascondendo i dettagli della comunicazione remota e dell'implementazione della **service procedure**.

Il **client program** chiama una RPC tramite il **client STUB**:

1. Serializza i parametri (*marshalling*).
2. Invia la richiesta al server tramite il **modulo di comunicazione**.
3. Riceve e deserializza la risposta (*unmarshalling*).
4. La restituisce al **client program**.

Dispatcher

Nel server ci sono molte procedure che possono essere eseguite.

Il **dispatcher** è responsabile di gestire le richieste ricevute dal **modulo di comunicazione** e instradarle verso la corretta **service procedure**.

Spesso è una componente dello **skeleton**.

Esso ispeziona il codice della richiesta e decide quale codice (*quale funzione dello skeleton*) eseguire per rispondere a tale richiesta.

Server STUB (Skeleton)

1. Prende una sequenza di byte dal **modulo di comunicazione**.
2. Ricava parametri → converte nel linguaggio locale (*un-marshalling*) della **service procedure**.
3. Invia i parametri alla **service procedure**.
4. Riceve risposta → Serializza (*marshalling*).
5. Invia la sequenza di byte al **modulo di comunicazione**.

Interface Definition

Specifica la firma (*interfacce*) delle funzioni: nomi, parametri, tipi di dati e valori restituiti.

Agisce come un contratto tra client e server, assicurando che entrambi abbiano una visione coerente delle funzioni disponibili.

Ci sono linguaggi detti **IDL** (*Interface Definition Language*) che permettono di definire queste interfacce.

Dai file IDL vengono generati **automaticamente** i codici del **client STUB** e **server STUB**.

Questa generazione viene fatta considerando anche i linguaggi con cui sono stati scritti il **client program** e la **server procedure**, perché i parametri, tra linguaggi diversi, vengono mappati in modo diverso.

Funzionamento RPC

1. **Programma client**: chiama una RPC utilizzando il codice esposto dal **client STUB**.
2. **Client STUB: MARSHALLING**: Serializza i parametri e invia una richiesta al server tramite **modulo**.
3. **Modulo di comunicazione (client)**: Trasmette la richiesta al server.
4. **Modulo di comunicazione (server)**: Riceve e passa al **server STUB** e al **dispatcher**.
5. **Dispatcher**: Rileva l'operazione desiderata e invia un valore al **server STUB**.
6. **Server STUB: UN-MARSHALLING**: Deserializza i parametri e invoca la funzione (*in base a ciò che ha ricevuto dal dispatcher*) sul **programma server**.
7. **Programma server**: Calcola il risultato e restituisce al **server STUB**.
8. **Server STUB: MARSHALLING**: Serializza e invia al **modulo**.
9. **Modulo di comunicazione (server)**: Invia la risposta al client.
10. **Modulo di comunicazione (client)**: Riceve e la passa al **client STUB**.
11. **Client STUB: UN-MARSHALLING**: Deserializza e restituisce al **programma client**.

Che cos'è una RMI?

Si basa su un'architettura di tipo **client-server**.

RMI → Invocare metodi di un oggetto remoto e ottenere il risultato.

CORBA → **middleware** standard che consente a oggetti scritti in linguaggi diversi (Java, C++, Python, ecc.) di interagire su piattaforme diverse come se fossero parte dello stesso sistema. Usa le IDL per definire le interfacce.

Java RMI: Differentemente da CORBA, non affrontiamo il problema dell'eterogeneità dei linguaggi, poiché **RMI è una tecnologia Java**, basata sulle interfacce remote.

Qual è l'obiettivo di RMI?

Permettere a oggetti Java di **comunicare tra loro come se fossero nella stessa macchina**, sfruttando la serializzazione.

Funzionamento dei riferimenti remoti relativamente alle RMI

Problemi: **localizzare** l'oggetto remoto, **chiamare il metodo** e restituire il risultato.



I problemi vengono risolti tramite i riferimenti remoti.

Riferimento remoto → permette l'accesso ad un oggetto remoto come se fosse nel nodo corrente.

Un riferimento remoto è **unico in tutto il sistema distribuito** ed è creato dal **RRM** (*Rem. Ref. Module*) o RMI Registry.

La struttura di un riferimento remoto è quella mostrata nell'immagine sotto:

32 bits	32 bits	32 bits	32 bits	
IP Address	Port	Creation time	Object nr.	Interface of r. obj

Bisogna poter identificare la macchina remota (**IP + porta**), **data di creazione**, il **numero di oggetto** e infine l'**interfaccia dell'oggetto remoto**.

Oggetto remoto e Interfaccia remota

Oggetto Remoto → Oggetto che implementa un'interfaccia remota.

Interfaccia Remota → Interfaccia che estende `java.rmi.Remote`.

`java.rmi.Remote` → **Interfaccia “marcatore”** (serve solo per caratterizzare qualcosa, in quanto non ha metodi), serve quindi per classificare un'interfaccia come remota.

Classificandola come remota, la JVM la tratterà in modo diverso, per questo `java.rmi.Remote` è detta “marcatrice”.

```
public interface RemoteInterface extends Remote {  
    String sayHello() throws RemoteException;  
}
```

Ogni dichiarazione di metodo in un'interfaccia remota (o nelle sue super-interfacce) deve essere una dichiarazione di metodo remoto quindi che utilizza un riferimento remoto.

Creazione di un oggetto remoto

1. Il server **crea un'istanza dell'oggetto remoto**.

Ricorda: L'oggetto remoto è un'istanza di una classe che **implements** un'interfaccia remota.

2. L'oggetto remoto **viene esportato**, in due modi:
 - **Implicitamente**, estendendo `UnicastRemoteObject`.
 - **Esplicitamente**, con `UnicastRemoteObject.exportObject(obj, port)`.

```
public class RemoteImplementation  
    extends UnicastRemoteObject // Esportazione implicita.  
    implements RemoteInterface {  
    // Oggetto remoto, poiché implementa un'interfaccia remota.  
    protected RemoteImplementation() throws RemoteException {  
        super(); // Esporta automaticamente l'oggetto  
    }  
    @Override  
    public String sayHello() throws RemoteException {  
        return "Hello";  
    }  
}
```

Il riferimento remoto viene ufficialmente creato al momento dell'esportazione.

3. L'oggetto remoto viene registrato dal server nel Registro RMI con `Naming.rebind("nome", oggettoRemoto)`.

Registrare → Associare un nome all'oggetto remoto e renderlo individuabile dai client.

4. Il client ottiene il riferimento remoto dal Registro RMI con `Naming.lookup("rmi://host/nome")`.

5. Il **Registro RMI** (Registro degli oggetti remoti) restituisce lo stub dell'oggetto remoto registrato sotto quel nome.
6. Lo stub viene serializzato e inviato al client.
7. Il client riceve lo stub e lo usa per invocare metodi remoti.

Funzionamento COMPLETO di una RMI

Interfaccia remota **Calculator**, notare che estende l'interfaccia **Remote**.

```
public interface Calculator extends Remote {  
    int add(int a, int b) throws RemoteException;  
}
```

Implementazione dell'oggetto remoto (**server procedure**):

```
public class CalculatorImpl extends UnicastRemoteObject  
    implements Calculator{ // Implementa un'interfaccia remota.  
  
    protected CalculatorImpl() throws RemoteException {  
        super(); // Esporta l'oggetto remoto automaticamente  
    }  
    public int add(int a, int b) throws RemoteException {  
        return a + b; // Metodo offerto al client.  
    }  
}
```

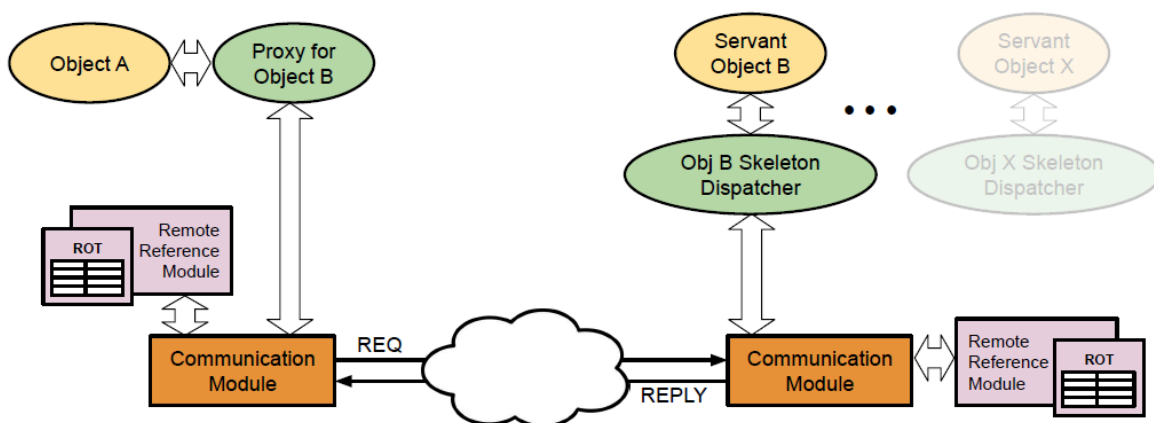
Ogni metodo remoto deve includere **java.rmi.RemoteException** nella sua clausola **throws**.

Codice client che interagisce con l'oggetto remoto (**client program**):

```
public class CalculatorClient {  
    public static void main(String[] args) {  
        // Prelievo dello STUB e lo rende disponibile per l'uso locale.  
        Calculator c = (Calculator)  
            Naming.lookup("rmi://localhost/CalculatorService");  
        int r = c.add(5, 3); // Chiamata della RMI.  
    }  
}
```

Codice server per registrare l'oggetto remoto:

```
public class CalculatorServer {  
    public static void main(String[] args) {  
        // Viene impostata la code base.  
        // Dove trovare i .class da far scaricare al client.  
        System.setProperty(  
            "java.rmi.server.codebase",  
            "http://localhost/classes/"  
        );  
        // Avvia il registro RMI sulla porta 1099.  
        Registry registry = LocateRegistry.createRegistry(1099);  
        // Crea l'oggetto remoto.  
        Calc c = new CalculatorImpl();  
        // Crea lo STUB da inviare al client.  
        Calculator stub = (Calculator)UnicastRemoteObject.exportObject(c, 0);  
        // Registra la RMI nel registro RMI.  
        registry.rebind("CalculatorService", stub);  
    }  
}
```



1. Il server crea un'istanza di `CalculatorImpl`.
2. `export()` → Viene generato il **client STUB**.

Quando un oggetto remoto **estende** `UnicastRemoteObject`, ottiene automaticamente il comportamento necessario per essere esportato come oggetto remoto. Questo semplifica il processo di pubblicazione e gestione delle chiamate RMI → non serve chiamare `export`.

3. Il server registra lo stub nel **Registro RMI** con `Naming.rebind("CalculatorService", client_stub)`, rendendolo accessibile ai client.
 - Registra l'oggetto nella **ROT** per poter ricevere richieste.
 - Quando un client fa `lookup()`, il riferimento remoto viene serializzato e inviato come stub.
4. Il client chiede al Registro RMI l'oggetto remoto con `Naming.lookup("rmi://localhost/CalculatorService")`.

5. Il **client program** riceve il **client STUB** (**calculator**) che rappresenta l' oggetto remoto (*proxy*).
 - Quando il client riceve lo stub tramite **lookup()**, RMI runtime lo inserisce nella **ROT**.
 - Questo permette di ottimizzare successive chiamate remote (evitando di reiniziare connessioni).

client STUB è una classe generata automaticamente da Java RMI che implementa l'interfaccia remota (*non serve avere la vera implementazione, la quale rimane nascosta nel server*) e si occupa di: serializzare, inviare la richiesta al server, attendere la risposta e deserializzarla.

Il client è convinto di avere il vero oggetto in locale (*dato che sia il proxy che il vero oggetto implementano la stessa interfaccia*).

client STUB è memorizzato (*dal client*) insieme alle altre classi del programma (*vediamo dopo*).

6. Il **client program** invoca **calculator.add(5, 3)**, il **client STUB**:
7. Il **client STUB** serializza i parametri della chiamata (5 e 3).

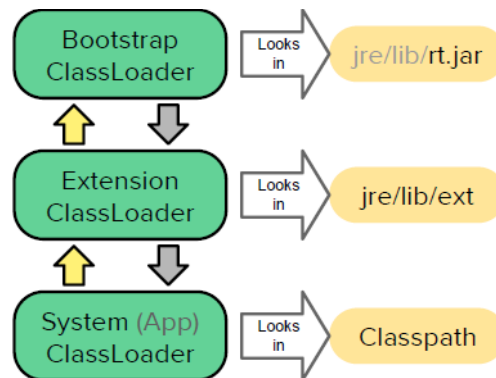
In RMI, per i metodi remoti di oggetti remoti, gli argomenti sono passati solo per valore e possono essere di tipo IN, il valore di ritorno è l'unico OUT.

8. Il **client STUB** invia la richiesta al server e **attende** la risposta.
9. Il server riceve la richiesta attraverso lo **skeleton** (**dispatcher**).
10. **skeleton** deserializza i parametri della chiamata (5 e 3).
11. **skeleton** individua l'oggetto remoto **CalculatorImpl** corretto nella **ROT**.
12. **skeleton** chiama **CalculatorImpl.add(5, 3)** nella **server procedure**.
13. **skeleton** serializza il risultato e lo invia indietro al client.
14. **client STUB** riceve la risposta dal server, deserializza il valore e lo restituisce al **client program**.

Class loading in JVM, differenze con applicazioni distribuite?

Class Loader Non Distribuito

I class loaders della JVM sono 3 e sono organizzati in una gerarchia basata sul delegare il lavoro al padre: ogni classloader chiede prima al “*padre*” di caricare la classe, nel caso non potesse allora proverà a fare il lavoro da solo.



1° → Bootstrap ClassLoader

Il primo ad agire.

Il modulo più alto (*bootstrap loader*) è parte della JVM, ha il compito di guardare in `jre/lib/rt.jar` che contiene le **classi fondamentali** (`String`, `Object`, `Integer`, ...).

Se la classe non è trovata, allora interviene il figlio.

2° → Extension ClassLoader

Carica le **librerie di estensione**, come quelle di crittografia o API opzionali.

3° → System(App) ClassLoader

Carica le **librerie dell'applicazione**, cioè il codice scritto dallo sviluppatore e le librerie aggiunte dall'utente.

Perché ci sono 3 ClassLoader? Non basterebbe averne 1?

Per tutelare le librerie fondamentali di Java e quindi proteggere la JVM. Inoltre ogni classloader cerca in un posto diverso.

1. Sicurezza:

Separare le classi core da quelle dell'applicazione impedisce al codice utente di compromettere la JVM.

2. Isolamento:

Evita conflitti di versioni. Ad esempio, un'applicazione può usare una versione diversa di una libreria senza interferire con le librerie del sistema.

Proprietà di "loading": unicità, visibilità.

Caricamento esplicito: `cl.loadClass(...)`, `Class.forName(...)`.

Class Loading distribuito

Il **class loading distribuito** è un meccanismo che consente a un'applicazione (*client*) di caricare classi da fonti remote (*server*) durante l'esecuzione.

Questo viene gestito da un classloader distribuito che si occupa di **localizzare e scaricare la definizione della classe necessaria**.

Quando si usa una RMI, il distributed classloader consente di **scaricare la definizione della classe** (il file `.class`) da un server, se il client non ha la definizione della classe di un oggetto remoto (il file `.class`).

Quando interviene?

Se il client riceve un'istanza di una classe che non ha nel proprio classpath.

- Se lo stub è stato generato dinamicamente con dynamic proxies (`java.lang.reflect.Proxy`) e il client non ha il codice specifico.
- Se il client non trova la classe dello stub, cerca nel `java.rmi.server.codebase`, un URL configurato dal server per distribuire le classi richieste.

Codebase → URL che punta alla posizione delle classi necessarie per il client.

```
System.setProperty("java.rmi.server.codebase", "http://example.com/classes/");
```

La codebase viene utilizzata quando il client riceve un riferimento remoto e il runtime RMI deve caricare dinamicamente lo stub.

1. Il server registra l'oggetto remoto nel Registry, specificando la codebase.
2. Il client chiama `Naming.lookup()`, che ottiene lo stub.
3. Se il client non ha la classe dello stub, il runtime RMI cerca nella codebase.
4. Il **RMIClassLoader** scarica la classe dallo specificato URL.
5. La classe viene caricata e resa disponibile al client.

Gli approcci per fare una Garbage Collection Locale?

Fondamentale per Java → Java non richiede di liberare la memoria (*come in C++*).

Processo **automatico** con cui la JVM libera la memoria non più utilizzata, eliminando oggetti inutili.

Non deve bloccare il sistema → "*interleaved*" con il programma principale.

Ci sono due approcci: Mark&Sweep e Reference Counting.

Mark&Sweep vs Reference Counting, come adattarli a sistemi distribuiti?

Mark&Sweep

♦ Fase 1: Mark (Marcatura)

1. Il **GC esamina tutti gli oggetti in memoria** e marca quelli raggiungibili partendo dagli oggetti radice (*variabili del metodo main, variabili statiche della classe main*).
2. Gli oggetti **non raggiungibili** non vengono marcati.

Grafo degli oggetti per determinare quelli raggiungibili.

Processo Lento → Il grafo **non è mantenuto costantemente aggiornato** ad ogni istante, ma viene costruito da zero quando il GC viene chiamato, quindi, quando esegue il **marking**.

Un oggetto **X è raggiungibile da Y se esiste un riferimento attivo da Y a X.**

Un **riferimento attivo** tra due oggetti **X** e **Y** in Java è presente quando **esiste un puntatore (reference)** da **Y** a **X** che può ancora essere usato dal programma.

```
public class ClassY {  
    public void MyMethod() {  
        MyObject X = new MyObject();  
        // L'oggetto X diventa raggiungibile da Y.  
        ...  
        X = null;  
        // L'oggetto X non è più raggiungibile da Y.  
    }  
}
```

♦ Fase 2: Sweep (Raccolta)

3. Il GC va in esecuzione, scansiona la memoria e **dealloca** gli oggetti **non marcati**.

📌 Vantaggi

- **✓ Gestisce i riferimenti ciclici** → Identifica gli oggetti realmente raggiungibili.

📌 Svantaggi

- **✗ Non è deterministico** → Memoria liberata solo quando il GC viene attivato.
- **✗ Prestazioni:** Traversare il grafo dei riferimenti è oneroso.

Reference Counting

Ogni oggetto ha un **contatore di riferimenti** che indica quante variabili lo puntano.

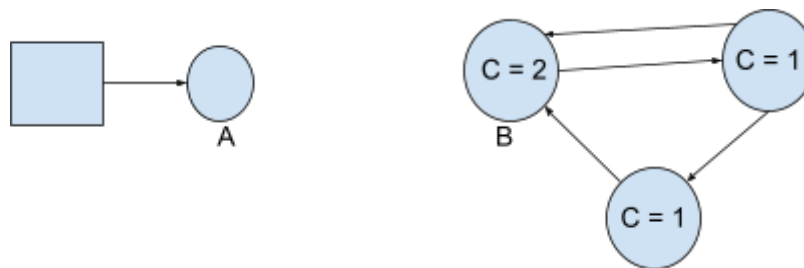
- Una variabile fa riferimento all'oggetto → contatore **aumenta**.
- Riferimento rimosso → contatore **diminuisce**.
- Se il contatore arriva a **0** → deallocato.

📌 Vantaggi

- **✓ Deallocazione immediata** → No fase di ricerca degli oggetti inutilizzati.
- **✓ Deterministico** → Memoria liberata appena l'oggetto non è più usato.

📌 Svantaggi

- **✗ Non gestisce riferimenti ciclici** → Se due oggetti si riferiscono a vicenda, il contatore non diventa mai zero → memory leak.
- **✗ Overhead di aggiornamento** → Ogni modifica ai riferimenti comporta operazioni di incremento/decremento.



Il GC locale è solitamente implementato utilizzando approcci Mark&Sweep.

Oggetti diversi mostrano diversi modelli di utilizzo (*in genere, si osserva un'elevata "mortalità infantile"*), e questo influisce sulle prestazioni del sistema perché in caso venissero chiamati di nuovo occorre caricarli da zero.

Per questo motivo, gli oggetti sono divisi in "popolazioni" (*ad esempio, giovani e anziani*) in base al loro comportamento, in modo da ottimizzare il processo complessivo (*solo gli oggetti più giovani sono impostati su bianco*).

Come determinare se un oggetto appartiene ad una popolazione o ad un'altra? All'inizio un oggetto appartiene alla popolazione giovane, se sopravvive per un tempo prestabilito verrà spostato in un'altra popolazione.

L'algoritmo (*ce ne sono vari, ma ne vedremo solo uno*) per l'attraversamento dell'intero grafo è detto **tri-color marking**.

Spiega l'algoritmo Tri-color marking.

Questo algoritmo segue l'approccio **Mark and Sweep** e non richiede lo stop dell'esecuzione del programma.

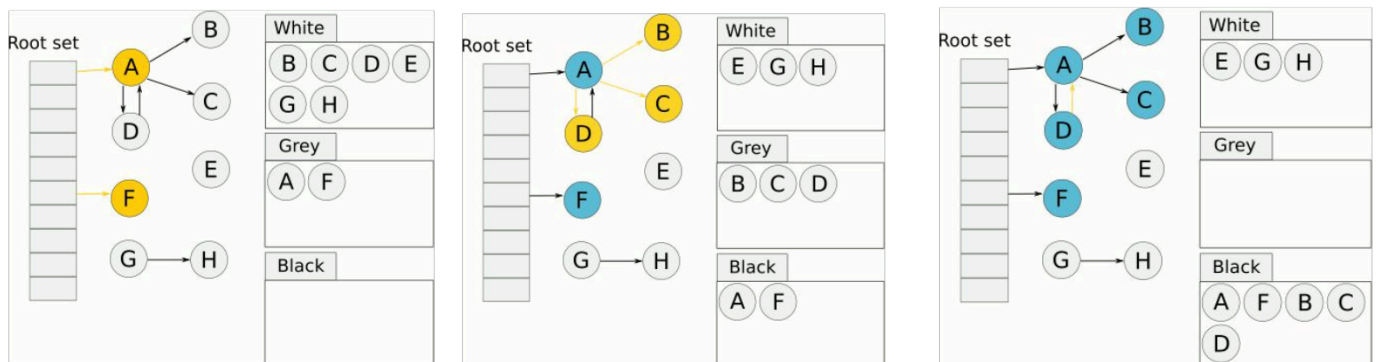
Mark

Divide gli oggetti in tre set:

- **Grigio**: Oggetti raggiungibili dalle radici e non ancora scannerizzati.
 - Inizialmente, contiene gli oggetti raggiungibili dagli oggetti radice.
- **Nero**: Oggetti non eliminabili.
 - Inizialmente vuoto.
 - Non hanno riferimenti da oggetti bianchi.
- **Bianco**: Oggetti candidati da eliminare.
 - Inizialmente, contiene tutti gli altri oggetti che non sono potuti andare in grigio.

Sweep

1. **While**: Finché il set grigio non è vuoto:
 - a. Prendiamo **X** grigio e spostiamolo in nero.
 - b. Muoviamo in grigio gli oggetti bianchi riferiti da **X**.
2. Se grigio vuoto → bianchi eliminati.



GC distribuita (problematiche e soluzioni)

Garbage collection per gli oggetti remoti, per lo scopo in genere si usa il **reference counting**.

Il client informa sempre il server sui riferimenti remoti che sta utilizzando (*perché li richiede attraverso la `Naming.lookup()`*).

Il server mantiene, per ogni entrata della ROT, la lista di client che hanno riferimenti remoti ad un certo oggetto remoto (detto *servant* in questo contesto).

Questa lista deve essere aggiornata ogni volta che un riferimento remoto è creato, duplicato o rimosso.

Le funzionalità del DGC sono specificate in una **interfaccia Java** chiamata:

```
java.rmi.dgc.DGC
```

Con due metodi:

```
dirty(<list of rem refs>, ... , <Client JVM_ID>)
```

Serve per informare il server che quel client JVM utilizza quella lista di riferimenti remoti.

```
clean(<list of rem refs>, ... , <Client JVM_ID>)
```

Serve per informare il server che quel client JVM non utilizzerà più i riferimenti remoti per un certo oggetto.

I due problemi che nascono dal DGC sono:

1. **Data race** tra `dirty(X , ... )` e `clean(X , ...)` da due client diversi.

Quando la lista dei detentori dell'oggetto `X` è composta da un singolo elemento (*se arriva prima una o l'altra ha effetti opposti*).

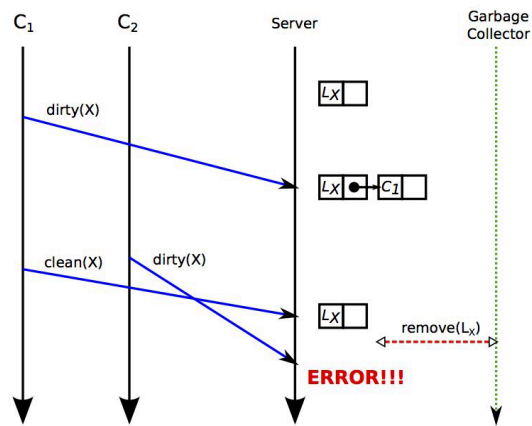
2. **Crash dei client** possono prevenire che una lista diventi vuota, portando ad un **memory leak**. Poiché il client non è in grado di dire che non userà un certo oggetto e quindi il server è obbligato a tenere quell'oggetto lì per sempre, portando ad una perdita di memoria a lungo termine.

Problema 1 - `clean(X, ...)` and `dirty(X, ...)` race

Il server ha un oggetto remoto `X` e C1, C2 lo vogliono utilizzare.

All'inizio il server ha la lista (*vuota*) corrispondente ai riferimenti remoti di `X`.

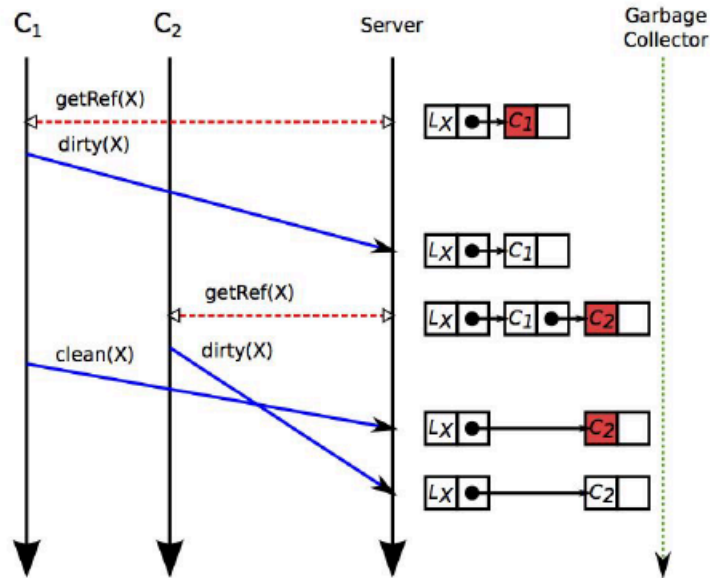
1. C1 invia `dirty(X)`.
2. Arriva `dirty(X)` di C1, il server aggiorna (*append*) la lista degli owner di `X`.
3. C2 invia `dirty(X)` e C1 invia `clean(X)`.
4. Arriva `clean(X)` di C1, il server rimuove C1 dalla lista degli owner di `X`.
5. La lista degli owner di `X` è vuota → `X` viene rimosso dal GC.
6. Arriva `dirty(X)` di C2 → **Eccezione** perché `X` è stato rimosso.



Una **soluzione** è quella dei **placeholder**.

Se C2 vuole fare una `dirty(X)`, significa che C2 conosce l'esistenza di `X` e quindi ha potuto prenderne il riferimento da qualche parte.

Facciamo in modo che il server dia la ownership di `X` nel momento in cui ne viene chiesto il riferimento.



Quando C2 chiede il riferimento remoto, viene creato un **placeholder** e messo nella lista.

Il **placeholder** elenca i nodi, che prima o poi invieranno un **dirty(X)**.

1. C1 invia **getRef(X)**, e viene messo un **placeholder** per C1.
2. C1 invia **dirty(X)**.
3. Arriva **dirty(X)** di C1, il server sostituisce il **placeholder** con la ownership.
4. C2 invia **getRef(X)**, e viene messo un **placeholder** per C2.
5. C2 invia **dirty(X)** e C1 invia **clean(X)**.
6. Arriva **clean(X)** di C1, il server rimuove C1 dalla lista degli owner di X.
7. Lista di X vuota → X NON rimosso, c'è il **placeholder** di C2.
8. Arriva **dirty(X)** di C2, il server sostituisce il **placeholder** con la ownership.
9. C2 invia **clean(X)**.
10. Arriva **clean(X)** di C2, il server rimuove C2 dalla lista di X.
11. Lista di X vuota → X rimosso.

Problema 2 - Crash del client

Supponiamo che:

1. C1 invia `getRef(X)`, e viene messo un **placeholder** per C1.
2. C1 invia `dirty(X)`.
3. Arriva `dirty(X)` di C1, il server sostituisce il **placeholder** con la vera ownership.
4. C1 termina (*crash*) e non invia `clean(X)`.

X non verrà mai cancellato dal GC → Memory leak.

Possiamo cambiare la semantica di `dirty(X)` in modo che il client indichi al server per quanto tempo utilizzerà una risorsa (*leasing*).

Adesso `dirty()` restituisce un tempo di lease, per cui rimarrà la ownership.

Il client può rinnovare il tempo inviando di nuovo una `dirty()`.

1. C1 invia `getRef(X)`, e viene messo un **placeholder** per C1.
2. C1 invia `dirty(X)`.
3. Arriva `dirty(X)` di C1, il server sostituisce il **placeholder** con la vera ownership.
4. Il server risponde a C1 con la durata della ownership.
5. Allo scadere, C1 invia `dirty(X)` per rinnovarla.
6. Il server risponde a C1 con la durata della nuova ownership.
7. C1 termina (*crash*).
8. Allo scadere del leasing la ownership di C1 viene rimossa da X.
9. La lista degli owner di X è vuota → X viene rimosso dal GC.

Qual è la differenza tra i Big Web Services e i REST Web Services?

Big Web Services e REST Web Services sono due **approcci** per costruire un servizio web di grandi dimensioni.

Le WS sono app client/server che comunicano tramite HTTP.

♦ Big Web Services (SOAP-based Web Services)

Seguono standard W3C definiti da **WS** come (*WS-Security*, *WS-Addressing*, ecc.).

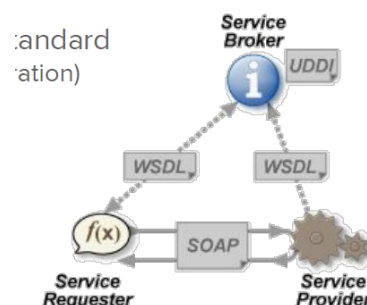
Approccio adatto quando è necessaria **sicurezza avanzata, transazioni distribuite e standardizzazione rigorosa**.

📌 Caratteristiche:

- ☒ **Protocollo** → Basato su **SOAP**.
- ☒ **Formato Dati** → Messaggi **XML** con struttura rigida e standard.
- ☒ **Descrizione API** → **WSDL (Web Services Description Language)** descrive in modo formale le operazioni disponibili in un documento XML.
- ☒ **Indipendente dal protocollo** → Può usare HTTP, SMTP, JMS, ecc.
- ☒ **Sicurezza avanzata** → Grazie a WS-Security.
- ☒ **Transazionalità e affidabilità** → Transazioni distribuite garantite.
- ☒ **Comunicazione Stateless o Stateful**.

📌 Svantaggi:

- ☒ **Pesantezza** → Messaggi XML verbosi → più latenza e consumo di banda.
- ☒ **Complessità** → Necessita di strumenti come WSDL.
- ☒ **Performance** → La serializzazione/deserializzazione di XML è più lenta rispetto a quella con JSON.



♦ 2. REST Web Services (RESTful APIs)

Si basano sull'architettura **REST (Representational State Transfer)** e utilizzano principalmente il protocollo **HTTP**.

📌 Caratteristiche:

- ☒ **Basato su risorse** → entità sono **risorse** identificate da un URL.
 - `http://example.com/users/123` rappresenta un utente con ID **123**.
- ☒ **Protocollo: HTTP:**
 - **GET** → Recupera dati.
 - **POST** → Crea risorsa.
 - **PUT** → Aggiorna risorsa.
 - **DELETE** → Elimina risorsa.
- ☒ **Formato Dati:** → **JSON**, XML, YAML, ecc.
- ☒ **Descrizione API** → **OpenAPI (Swagger)** o **anche nessuna**.
- ☒ **Più leggero e veloce** → Meno overhead rispetto a SOAP grazie a JSON e assenza di WSDL.
- ☒ **Stateless** → Ogni richiesta contiene tutte le informazioni necessarie, senza mantenere lo stato lato server.
- ☒ **Cacheability** → Le risposte possono essere contrassegnate come *"cacheable"* per migliorare le prestazioni.
- ☒ **REST Uniform Interface** → L'interazione tra client-server avviene tramite un'interfaccia uniforme, semplificando la comunicazione.
- ☒ **Sistema a strati** → Un client non può distinguere il server da un intermediario.

📌 Svantaggi:

- ☒ **Manca supporto nativo per sicurezza avanzata** → Obbligo di HTTPS per autenticazione e sicurezza.
- ☒ **Meno adatto a transazioni complesse** → Non supporta direttamente WS-AtomicTransaction.
- ☒ **Meno standardizzato** → Ogni API può avere un design diverso, senza un linguaggio formale come WSDL.

📌 Usato in:

- ♦ Applicazioni web, mobile, micro servizi, IoT, servizi pubblici su Internet.

CAPITOLO 11 - Enterprise Java Beans

Cosa è un Java Servlet e come funziona?

Un **Servlet** è una **classe Java** che gestisce le richieste HTTP per un certo URL e produce risposte dinamiche all'interno di una applicazione web.

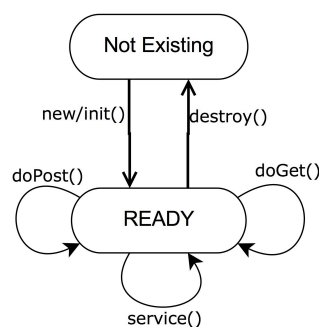
È una delle componenti fondamentali di **Jakarta EE** (*Java Enterprise Edition*) per lo sviluppo di applicazioni web.

Jakarta EE è una piattaforma per lo sviluppo di **applicazioni enterprise** in Java, ovvero applicazioni su larga scala come sistemi aziendali, servizi web, e-commerce, ecc.

Funzionamento di un servlet

I Servlet sono eseguiti all'interno di un **container servlet** (come *Tomcat*), che ne gestisce il ciclo di vita e l'esecuzione.

I **Servlet** (sia in *Jakarta EE* che in *Tomcat*) **di default** adottano il modello **thread-per-request**, ma possono essere configurati per usare altri modelli (come il **thread pooling**) di gestione della concorrenza.



1. **Caricamento e istanziazione:** Ricevuta richiesta HTTP per una risorsa.

La risorsa è mappata con un URL e da qualche parte (*file di configurazione*) è presente l'associazione con una certa classe Servlet definita dallo sviluppatore.

2. Il **container servlet** crea un'istanza del relativo servlet.
 - **Thread Pooling:** Il container assegna un thread dal pool per il servlet.
 - **Thread-per-Request:** Il container crea un nuovo thread per il servlet.
3. Una volta che il servlet è stato caricato, il container invoca il metodo `service()` del servlet per gestire la richiesta HTTP.

`service` funge da spartiacque dove viene scelto il metodo (*scritto dallo sviluppatore*) per gestire la richiesta (`doGet()`, `doPost()`, `doDelete()`, ecc).

`service(HttpServletRequest request, HttpServletResponse response)`

- `HttpServletRequest`: richiesta HTTP: parametri, cookie e metodo (*GET*, ecc.).
- `HttpServletResponse`: per inviare una risposta al client.

4. Elaborazione della richiesta.

Da `HttpServletRequest` il servlet estrae i dati e poi esegue logica scritta dallo sviluppatore.

Lo sviluppatore in genere scrive il codice della `doGet()` e `doPost()`.

`doGet` viene chiamato da `service` quando arriva una richiesta **HTTP GET**.

5. **Generazione della risposta:** Completata l'elaborazione, servlet invia una risposta tramite `HttpServletResponse`.
6. **Chiusura:** Il container invoca il metodo `destroy()` per distruggere il servlet.
 - **Thread Pooling:** Il thread viene rilasciato per servire nuove richieste.
 - **Thread-per-Request:** Il thread viene eliminato.

Modello Thread-per-Request

Di default, i servlet container **creano un nuovo thread** per ogni richiesta HTTP.

-  Semplice ed efficace per la maggior parte delle applicazioni.
-  Le richieste vengono servite in parallelo, migliorando la scalabilità.
-  La gestione dei thread è delegata al servlet container.
-  **Overhead dovuto al numero di thread:** Con richieste simultanee, il numero di thread può crescere troppo, causando overhead di gestione e consumo elevato di memoria (*risolto con thread pooling*).
-  **Concorrenza:** Un thread bloccato (es. I/O lento) può ridurre l'efficienza del sistema.

Mappatura tra Servlet e URL

Il container è responsabile della gestione del ciclo di vita dei servlet e della mappatura degli URL ai servlet corrispondenti.

Ogni servlet ha i suoi metodi `doGet` e `doPost`.

La configurazione dei servlet all'interno dell'applicazione web può essere effettuata in due modi diversi:

- **Mappatura Servlet-URL** tramite il file `WEB-INF/web.xml`:
 - `servlet-name` → `servlet-class`
 - `servlet-name` → `url-pattern`
- **Annotazioni** `@WebServlet` posizionate sopra la definizione della classe del servlet.

```
@WebServlet(  
    name = "MyAnnotatedServlet",  
    urlPatterns = {"/foo", "/bar", "/pippo*"}  
)  
  
public class MyServlet extends HttpServlet {  
    @Override  
    protected void doGet(HttpServletRequest req, HttpServletResponse res)  
        throws ServletException, IOException {  
        // Elaborazione della richiesta.  
        ...  
        // Generazione della risposta.  
        res.getWriter().println("Hello from doGet!");  
    }  
}
```

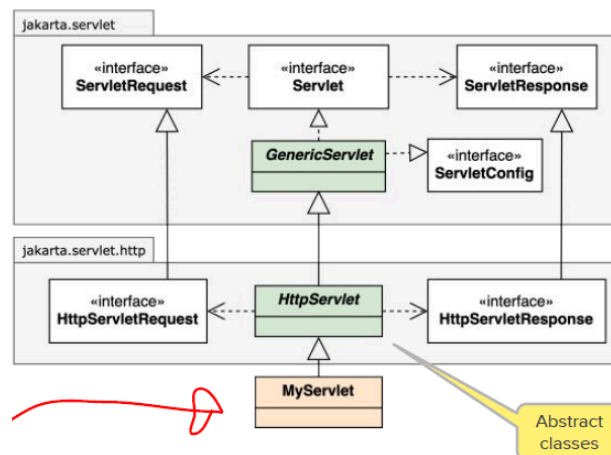
In Tomcat, la gestione ordinaria dei contenuti statici (*ad esempio, pagine html, immagini, ecc.*) viene eseguita utilizzando un servlet "di sistema" dedicato (*il sistema si comporta sempre allo stesso modo*):

DefaultServlet, che viene chiamato per qualsiasi URL REQ che fa riferimento a contenuti statici. Classe di implementazione:

`org.apache.catalina.servlets.DefaultServlet`, che estende la classe astratta `HttpServlet`.

Interfaccia di Servlet

Un servlet è una classe che implementa l'interfaccia `jakarta.servlet.Servlet`.



I servlet personalizzati devono estendere `GenericServlet` o, solitamente, `HttpServlet`.

Quante istanze di una classe Servlet ci sono in una singola webapp?

Di default, il container crea **UNA SOLA ISTANZA** di una classe servlet per tutta l'applicazione e viene condivisa da **tutte le richieste concorrenti** allo URL associato.

Viene caricata una sola volta in memoria dal **ClassLoader** del container quando l'applicazione web viene avviata.

Ogni richiesta viene eseguita in un **nuovo thread**, che chiama il metodo `service()` del servlet.

Dato che l'istanza della classe Servlet è unica, se necessario, servono meccanismi di sincronizzazione tra i thread che soddisfano le richieste.

Se 100 utenti fanno una richiesta al servlet `MyServlet`, verrà usata **un'unica istanza** di `MyServlet`, e ogni richiesta sarà gestita da un thread separato.

Ha senso tenere 10 oggetti Servlet per una specifica classe?

No, perché il container gestisce una **singola istanza** e la rende **thread-safe** attraverso l'uso di thread separati (*thread-per-request*) per le richieste.

Creare **più istanze manualmente** non è necessario e può causare problemi di gestione.

Richieste e risposte in un Servlet

Le richieste e delle risposte sono rappresentate come oggetti `HttpServletRequest` e `HttpServletResponse` della libreria `jakarta.servlet.http`.

1. Creazione degli Oggetti Richiesta e risposta

Il container servlet riceve una richiesta HTTP e costruisce i due oggetti.

2. Accesso ai Parametri di richiesta

Per recuperare dati dalla richiesta, `HttpServletRequest` fornisce i seguenti metodi:

- `getParameterNames()`: elenco dei nomi dei parametri.
- `getParameter(name)`: valore di un parametro specifico, identificato dal nome.
- `getParameterValues()`: array con tutti i valori di un parametro, utile se il parametro ha più valori (*come in un campo di tipo checkbox*).

3. Creazione del Corpo della risposta

Per inserire dati nella risposta, `HttpServletResponse` fornisce i seguenti metodi:

- `getWriter()`: restituisce un oggetto `PrintWriter` per inviare dati di tipo testo (HTML, JSON o XML).
- `getOutputStream()`: restituisce un oggetto `ServletOutputStream` per inviare dati di tipo binario (*file o immagini*).
- `setContentType(<MIME_type>)`: per impostare il **tipo** di contenuto della risposta.

Entrambi i flussi di output (`Writer` e `OutputStream`) devono essere chiusi dopo l'uso per liberare risorse.

JSP, per cosa sono usati? relazione con un Servlet?

I servlet risultano difficili da mantenere perché uniscono strettamente la logica di business (*scritta in Java*) e la presentazione (*tutta la parte relativa allo HTML e quindi alla GUI*).

Questo porta a un codice poco leggibile e difficile da modificare.

I JSP sono un modo per separare la generazione della presentazione (ovvero *del codice HTML della risposta*) dal codice del servlet.

Template

Il **templating** è un meccanismo per separare la logica dell'applicazione dalla presentazione, permettendo ai servlet di concentrarsi sulla gestione della logica e ai file di template (come *JSP o specifici framework di templating*) di occuparsi della visualizzazione.

Ecco un esempio di JSP, ovvero `welcome.jsp`:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Welcome Page</title>
  </head>
  <body>
    <h1>Benvenuto, ${username}!</h1>
  </body>
</html>
```

Dentro il template (*HTML*) possiamo inserire il codice utilizzando **testo speciale**.

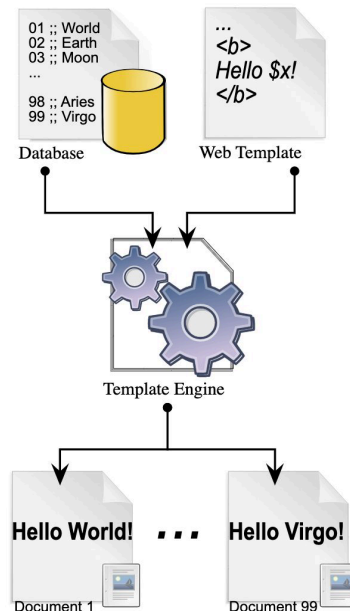
La logica viene inserita usando tag speciali `${}`, poi il motore di template processa questi file e genera contenuti dinamici.

```
@WebServlet("/welcome")
public class WelcomeServlet extends HttpServlet {
    @Override
    protected void doGet(
        HttpServletRequest request,
        HttpServletResponse response
    ) throws ServletException, IOException {
        // Imposta un attributo da passare alla JSP.
        request.setAttribute("username", "John Doe");
        // Creiamo un RequestDispatcher, per inoltrare la richiesta ad un altro Servlet.
        RequestDispatcher dispatcher =
            request.getRequestDispatcher("/WEB-INF/views/welcome.jsp");
        // La risposta HTTP è generata tramite il file JSP.
        // WelcomeServlet non scrive nulla in response, passa il controllo al JSP.
        dispatcher.forward(request, response);
        // La risposta viene inviata dal servlet generato dal JSP, non da WelcomeServlet.
        // Il browser non vedrà un cambio di URL → Trasparente al client.
    }
}
```

Questo template può essere processato da un engine di processazione dedicato. Molti sistemi di template sono sviluppati per generare contenuti customizzati

Come funziona la processazione di un JSP?

I JSP sono elaborati da un Template Engine il quale processa questi file e genera contenuti dinamici.

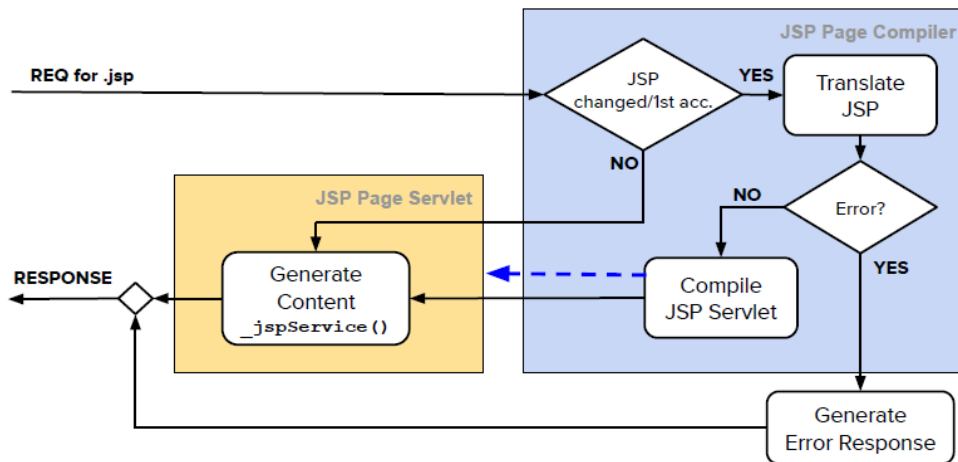


Il **template engine** è responsabile di:

1. Analizzare i placeholder **`$ { }`** nel template, che rappresentano dati o logica da processare.
2. Creare il documento finale basandosi su:
 - Una collezione di dati (*come un dataset*).
 - Eventuali logiche definite nel template stesso.

Questo schema di elaborazione può essere applicato a diversi livelli, come nella generazione dinamica di contenuti complessi o nella personalizzazione di interfacce utente.

JSP Page Servlet e JSP Page Compiler



Mostra come una richiesta HTTP per una pagina `.jsp` venga gestita, tradotta, compilata e servita come risposta dinamica.

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>
<html>
<head><title>JSP Example</title></head>
<body>
    <h1>Benvenuto!</h1>
    <p>Oggi è: <%= new java.util.Date() %></p>
    // Codice Java (scriptlet) che genera la data corrente.
</body>
</html>
```

1. Richiesta della pagina JSP

- Richiesta HTTP per una pagina `.jsp`, viene inoltrata al **JSP Page Compiler**.

2. Verifica dello stato della pagina JSP

- Il compilatore controlla se la pagina `.jsp` è stata modificata rispetto all'ultima compilazione o se si tratta del primo accesso alla pagina:
 - Sì** → Avvia il processo di traduzione (*lento*).
 - No** → Viene utilizzata la versione precedentemente compilata.

3. Traduzione della JSP

- Il file `.jsp` viene “convertito” in un servlet (*JSP Page Servlet*), ovvero in una classe Java che estende `HttpServlet`.

HTML + script JSP `<%= %>` → `_jspService()`.

4. Esecuzione del codice JSP (Servlet) dal container JSP.

- Richiesta → `_jspService()` invocato → genera il contenuto dinamico.

Il **Servlet generato dal JSP** rimane in esecuzione finché l'applicazione web è attiva nel **container Servlet**. Vediamo come funziona e quanto dura il suo ciclo di vita.

EJB e come vengono gestiti dall'Application Server, quali sono i tipi? come viene gestito il lifecycle

Gli **EJB (Enterprise Java Beans)** sono un tipo specifico di componente in **Jakarta EE** e offrono solo alcune **funzionalità specifiche**.

- **EJB** → Si occupa della **logica di business**, usato in **applicazioni enterprise**.
- **Servlet** → Gestisce richieste HTTP, usato nelle **applicazioni web**.

L'**Application Server** (es. GlassFish) gestisce automaticamente il loro **ciclo di vita**, **concorrenza**, **sicurezza e transazioni**.

Questo approccio consente di mantenere i **moduli client** più leggeri, poiché **gran parte della logica viene eseguita sul server**, alleggerendo il compito del programmatore che altrimenti dovrebbe costruire componenti per conto suo.

Riutilizzabilità: È possibile costruire nuove applicazioni riutilizzando EJB esistenti, il che facilita la manutenzione e lo sviluppo.

Tipologie di EJB

1. **Session Bean:** Gestisce la logica di business per le applicazioni distribuite.
 - **Stateless:** Non mantengono stato tra chiamate (es. servizi REST).
 - **Stateful:** Mantengono stato per un client specifico (es. carrello e-commerce).
 - **Singleton:** Una sola istanza condivisa in tutta l'app.

Quando un client invoca un metodo di un **Stateless EJB** o un **Singleton EJB**, il **container assegna un thread** per eseguire la richiesta.

2. **Message-Driven Bean (MDB)**
 - Asincroni, ricevono messaggi da JMS (Java Message Service).

Stateful Session Beans

Mantengono lo stato tra le invocazioni dei metodi, rendendo la sessione specifica per ciascun client.

```
@Remote // Interfaccia remota.  
public interface AccountSessionBeanRemote {  
    void deposit(double amount);  
    void withdraw(double amount);  
    double getBalance();  
}
```

E l'implementazione:

```
@Stateful  
public class AccountSessionBean implements AccountSessionBeanRemote {  
    private double balance = 0; // Stato del bean.  
    @Override  
    public void deposit(double amount) { balance += amount; }  
  
    @Override  
    public void withdraw(double amount) {  
        if (balance >= amount) balance -= amount;  
    }  
  
    @Override  
    public double getBalance() { return balance; }  
}
```

Questi bean conservano i valori delle variabili di istanza tra le chiamate del client.

La sessione con il bean non coincide necessariamente con la sessione web dell'applicazione.

Quando il client termina la sessione o il bean viene rimosso, lo stato viene perso.

Stateless Session Beans

Ogni invocazione è indipendente, rendendoli più scalabili e adatti per applicazioni con molti client.

Il server applicativo ottimizza mantenendo un **pool di session bean**: quando avviene una richiesta il primo session bean disponibile verrà utilizzato per fornire il servizio.

Sono più efficienti rispetto ai Stateful Session Beans in applicazioni con molti client.

Inoltre, possono implementare servizi web (*grazie al idempotenza*)

Singleton Session Beans

Esistono come un'unica istanza per applicazione, mantenendo lo stato tra le chiamate dei client, ma non tra i riavvii del server.

Il codice deve essere thread safe, in quanto unico per tutti i clienti.

Possono essere utilizzati per implementare endpoint di servizi web.

Message Driven Beans

Eseguono la logica di business in risposta a messaggi, piuttosto che a chiamate di metodi. Sono utilizzati per implementare l'**elaborazione asincrona**.

Ci riferiamo alle sessioni non relative solo alle applicazioni web, la sessione è relativa alla conversazione tra due componenti non necessariamente passando attraverso una connessione web.

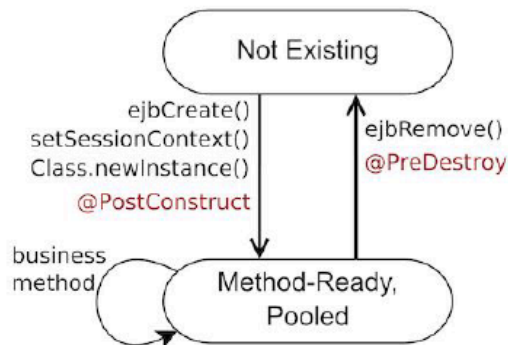
```
public interface OrderProcessor { void processOrder(String order); }
@MessageDriven(
    mappedName = "java:/jms/queue/OrderQueue", // Nome della coda JMS
    activationConfig = {
        @ActivationConfigProperty(
            propertyName = "destinationType",
            propertyValue = "javax.jms.Queue"
        )
    }
)
public class OrderMDB implements OrderProcessor {

    @Override
    public void processOrder(String order) {
        // Logica di business per elaborare l'ordine
    }

    // Quando un messaggio è ricevuto
    public void onMessage(Message message) {
        try {
            if (message instanceof TextMessage) {
                String order = ((TextMessage) message).getText();
                processOrder(order); // Processiamo il messaggio
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Ciclo di vita di uno Stateless Session EJB

Il suo ciclo di vita è gestito completamente dall'**Application Server**, che utilizza un meccanismo di **pooling** per riutilizzare le istanze ed evitare overhead di creazione/distruzione.



1 Creazione 🛠️

L'Application Server crea un pool di istanze all'avvio o alla prima richiesta.

Il costruttore viene chiamato (**new**), seguito da eventuali metodi di inizializzazione (**@PostConstruct**).

2 Chiamata ai metodi del bean ⚡

Quando un client invoca un metodo dello EJB:

Il container **prende un'istanza dal pool** e la associa temporaneamente al client.

3 Restituzione al pool ↺

Dopo l'elaborazione, l'istanza viene **rilasciata nel pool** per essere riutilizzata.

Ciclo di vita di un Singleton Session EJB (not pooled)

1 Creazione 🛠️

Viene **creato una sola volta** quando l'applicazione si avvia. Rimane attivo fino alla chiusura dell'Application Server.

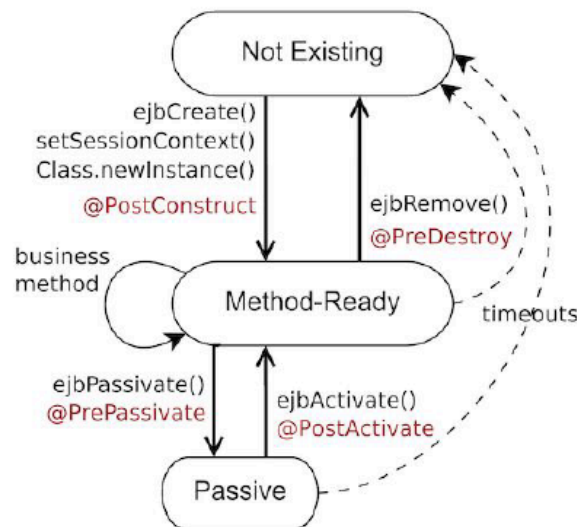
Il costruttore viene chiamato (**new**), seguito da eventuali metodi di inizializzazione (**@PostConstruct**).

2 Chiamata ai metodi del bean ⚡

3 Restituzione al pool ↺

Prima della distruzione, se presente, può essere invocato il metodo **@PreDestroy** per eseguire la pulizia finale.

Ciclo di vita di uno Stateful Session EJB



La prima parte dei due schemi (*quella in alto*) è circa la stessa, gli oggetti non sono raccolti in un pool come nel caso precedente.

Creazione

Quando un client invoca il suo metodo per la prima volta. L'Application Server crea un'istanza del bean, assegnandola al client.

Viene invocato costruttore ed eventuale `@PostConstruct`.

A questo punto, l'istanza è pronta per essere utilizzata dal client.

Utilizzo

Durante la sessione, il client può invocare metodi del bean in qualsiasi momento. Ogni chiamata al bean mantiene il **contesto e lo stato** dell'istanza.

Stato: Il bean può modificare variabili di istanza, e queste modifiche vengono conservate tra le chiamate (questo è il comportamento "stateful").

Le invocazioni successive del client utilizzano lo stesso oggetto bean, che conserva lo stato tra le chiamate.

Passivazione e Attivazione

Se l'istanza non è stata utilizzata per un certo periodo, l'Application Server può decidere di **passivarla**, ovvero salvarne lo stato su disco per liberare risorse. Quando il client richiede nuovamente il bean, l'istanza viene **riattivata** e il suo stato ripristinato.

- **Passivazione:** L'istanza è serializzata su disco.
- **Attivazione:** L'istanza viene deserializzata e riportata in memoria.

Rimozione

Chiusura della sessione: Il client invoca esplicitamente il metodo di rimozione `remove()`.

Questo metodo è legato all'interfaccia `Session` e viene invocato quando il client è pronto a terminare la sessione.

Timeout della sessione: Se il client non interagisce con il **bean Passivato** per un periodo di tempo definito (timeout di sessione), l'Application Server può decidere di **terminare la sessione** automaticamente, rimuovendo l'istanza del bean.

@PreDestroy (opzionale): Se definito, questo metodo viene invocato prima che l'istanza venga distrutta, per eseguire eventuali operazioni di pulizia.

CAPITOLO 12 - FUNDAMENTAL DISTRIBUTED ALGORITHMS

Wave algorithms

Classe di **algoritmi distribuiti** utilizzati per **diffondere (e/o raccogliere) informazioni** attraverso una rete e garantire che ogni nodo sia raggiunto prima della terminazione del processo.

Sono spesso impiegati in contesti come **elezione di un leader**, **esplorazione della rete** e **terminazione distribuita**.

Tipicamente la richiesta nasce da uno o più initiator, che poi, come un onda (*un'onda di informazioni*) coinvolge tutti gli altri nodi e ciascun nodo deve restituire le informazioni richieste.

- **Propagazione ad onda:** L'informazione (*partendo dall'initiator*) attraversa la rete in modo graduale.
- **Terminazione garantita:** L'algoritmo è finito, ovvero ha un numero finito di step e prima o poi di sicuro terminerà.
- **Eventi di decisione:** Contiene uno o più eventi di decisione in cui vengono prese decisioni basate sui dati raccolti.

Eventi di decisione

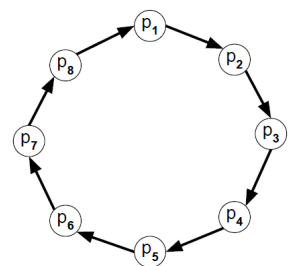
Quando un processo in un sistema distribuito **prende una decisione** (es. elezione di un leader).

Per ogni evento di decisione d e processo p_i , vale $c \xrightarrow{HB} d$ per un certo c in p_i .

Grazie alla relazione HB, possiamo stabilire **quali eventi influenzano la decisione**.

Esempio di algoritmo Wave - Anello

Supponiamo di avere una topologia ad anello come in figura, l'initiator può essere un p_x qualsiasi (*decentralizzato*).



Il codice initiator:

```
send(toNeighbor, OutToken) // Invia il token al vicino.
receive(fromNeighbor, inToken) // Riceve il token dal vicino.
// Se non ci sono messaggi nella coda, si mette in attesa.
decide() // Prende una decisione in base alle informazioni ricevute.
```

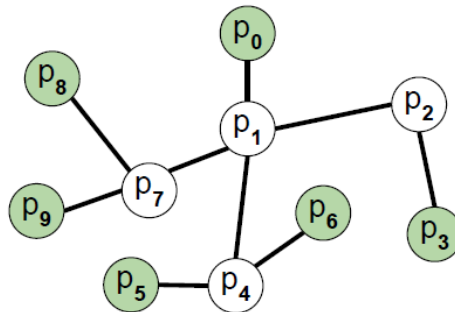
Il codice di un nodo non-initiator:

```
receive(fromNeighbor, inToken) // Riceve il token dal vicino.
doSomething() // Fa qualcosa con le informazioni ricevute.
send(toNeighbor, OutToken) // Invia il token al vicino.
```

Il messaggio è inviato e questo verrà propagato e ad un certo tempo tornerà indietro verso il nodo initiator (*completa un giro dell'anello*).

Esempio di algoritmo Wave - Tree Wave

I nodi sono organizzati in una topologia ad albero connesso senza loop.



L'initiator può essere una foglia qualunque (*decentralizzato*) e invia il token al proprio "padre".

L'algoritmo procede verso la radice, raccogliendo informazioni o propagando messaggi man mano che si sale nell'albero.

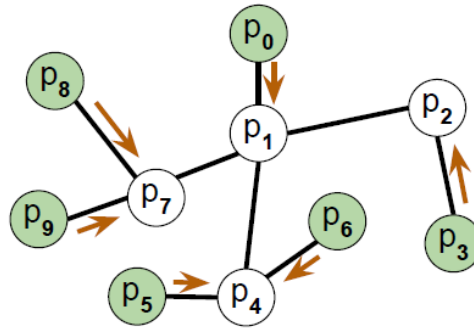
Il nodo foglia invierà direttamente il messaggio all'unico neighbor, dopo questo, il prossimo step è la ricezione di un messaggio dallo stesso neighbor.

```
bool recv[n_neigh] //array initialized [false]

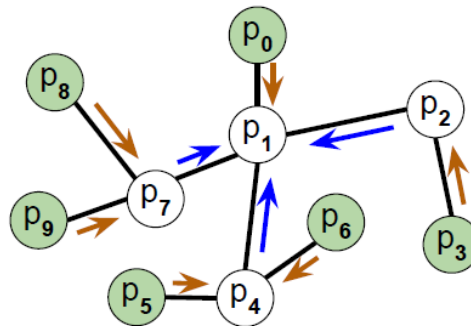
// loop executed n_neigh - 1 times
while len(filter(false, recv)) > 1:
    receive(fromNeighbor, inToken)
    recv[fromNeighbor] = true

//toNeighbor: the only proc with recv == false
send(toNeighbor, outToken)
receive(toNeighbor, inToken)
recv[toNeighbor] = true //not necessary...
decide()
//this way, only two processes decide. Possibly:
//foreach n in neighbors\toNeighbor:
//    send(n, outToken)
```

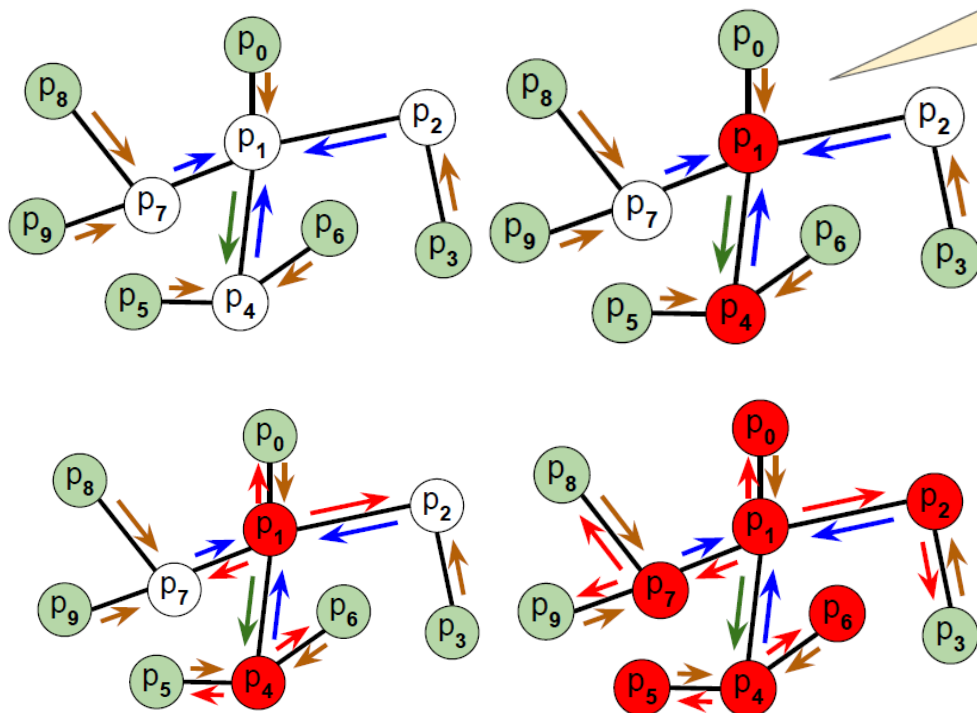
1. Ogni nodo non-foglia esegue per `#neighbours - 1` volte il ciclo `while` dove aspetta di ricevere un token da ogni figlio.
2. Non appena un messaggio viene ricevuto, l'entry corrispondente viene impostata a `true`.
3. Il loop termina quando avremo solo una posizione che vale `false`, corrispondente all'unico processo da cui non abbiamo ricevuto un messaggio (il padre).
4. Quel processo è il padre a cui gli viene inviato un messaggio.

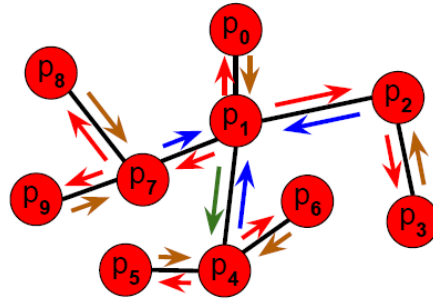


Nel **prossimo step** vediamo che p1 riceve i messaggi, sia p1 che p4 sono in grado di raggiungere il punto di decisione nel codice (*decide*).



E così via, tutti i nodi ricevono i messaggi.





Alla fine l'informazione è distribuita tra tutti i nodi.

Traversal algorithms

Un **algoritmo di traversamento** è un tipo di algoritmo a onda centralizzato dove l'iniziatore invia un token che attraversa tutta la rete, toccando ogni nodo.

Tutti i processi ricevono il token almeno una volta e infine torna all'iniziatore, che esegue un evento decisionale basato sulle informazioni raccolte.

Questi algoritmi vengono spesso utilizzati per esplorare reti, raccogliere dati o determinare lo stato globale di un sistema distribuito.

Algoritmi distribuiti: Ricart-Agrawala

Informazioni di base

Mutua esclusione: garantisce che solo un processo alla volta possa accedere a una risorsa condivisa.

L'accesso ad una risorsa condivisa avviene in una specifica sezione del codice detta "sezione critica" (**Critical Section, CS**).

Una sezione critica è delimitata dalle operazioni `req_access()` (richiesta di accesso) e `rel_access()` (rilascio della risorsa).

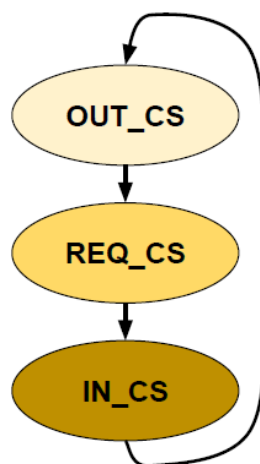
Safety e liveness

Un sistema ha delle proprietà che possono essere suddivise in **safety** e **liveness**.

- **Sicurezza (ME1):** Al massimo un processo può essere nella CS contemporaneamente.
- **Liveness (ME2):** Ogni richiesta di accesso o rilascio della CS verrà prima o poi soddisfatta (*non sappiamo quando un processo entrerà nella sezione critica, però sicuramente in futuro ci accederà*).

Stati di un Processo rispetto alla Sezione Critica

- **OUT_CS:** Il processo **non ha ancora richiesto di entrare** nella CS.
- **REQ_CS:** Il processo **ha richiesto l'accesso ma non è ancora entrato** nella CS.
- **IN_CS:** Il processo **è entrato** nella CS.



Tipologie di algoritmi di mutua esclusione

1. **Algoritmo con permessi Individuali:** Ogni processo concede il permesso di accedere alla CS solo per sé stesso.

2. **Algoritmi basati su Token:** Un unico token è condiviso tra tutti i processi e solo il quello che possiede il token può accedere alla CS.
3. **Algoritmo con permessi con Arbitro:** Esiste almeno un processo (*arbitro*) che risolve i conflitti tra le richieste. I permessi vengono restituiti all'uscita dalla CS.

Algoritmo Ricart-Agrawala

Questo algoritmo appartiene alla categoria dei **permessi individuali** e consente a un solo processo alla volta di accedere alla CS (ME1).

Si basa sull'invio di messaggi per sincronizzare i processi coinvolti (*ovvero interessati alla CS in questione*).

L'idea di base è la seguente: per entrare in una CS, un processo deve chiedere il permesso a tutti gli altri processi coinvolti mediante un messaggio **REQ** e aspettare il relativo **OK** da ogni altro processo.

Risoluzione dei conflitti

Ovviamente possono esserci conflitti con i **REQ** (*due processi lo mandano nello stesso momento*).

Risolviamo con i timestamp di Lamport (**totally-ordered**) e con i PID dei due processi.

Alla **REQ** viene inserito il timestamp e la richiesta con il timestamp minore ha priorità.

$$(t_{sa}, i) < (t_{sb}, j) \Leftrightarrow (t_{sa} < t_{sb}) \vee [(t_{sa} = t_{sb}) \wedge (i < j)]$$

In caso di timestamp sincronizzati, viene fatto un confronto dei PID.

Implementazione dell'algoritmo

Per inviare la **REQ**, deve essere eseguita la funzione:

```
procedure req_access() // pid: local pid
  mystate = REQ_CS
  pending_oks = N-1 //all the others
  myreq_lt = myclock+1 // 1.timest. for my req
  req.ts = (myreq_lt, pid) //set msg timest.
  foreach p in req set send(p, req)
  wait_until (pending_oks == 0 )
  mystate = IN_CS // then, CS
```

Semplificazione: la variabile interna **myclock** rappresenta il Lamport timestamp della richiesta del processo.

Se ci sono **n** processi, allora un processo deve collezionare **n-1 OK**.

Ogni volta che un **OK** viene ricevuto, la variabile **pending_oks** viene decrementata (*la variabile tiene conto degli OK che devono ancora essere ricevuti*).

La funzione **onReq** viene eseguita ogni volta che viene ricevuto un **REQ**.

Possono esserci diversi processi che richiedono l'accesso concorrentemente allo stesso tempo; per questo motivo nella funzione **onReq** il processo deve aggiornare il suo clock interno **myclock** secondo la regola dei timestamp di lamport.

```
procedure onReq(proc sender, msg m)
  myclock = max(myclock, m.ts)
  my_prior = (mystate != OUT_CS) and (myreq_lt,pid) < m.ts
  if not my_prior then send(sender, ok)
  else req_delayed.append(sender)
```

Se il processo ha una priorità minore rispetto ad altri, invia lo **OK** per avvertire chi ha una priorità maggiore.

Il processo invia la **OK** in una (**or**) di queste due condizioni:

- Ha già eseguito la CS, ovvero si trova nello stato **OUT_CS**.
- **Conflitto di REQ**: Se il processo ha una priorità minore (*quindi un timestamp maggiore*) $(t_{sa} < t_{sb}) \vee [(t_{sa} = t_{sb}) \wedge (i < j)]$

Altrimenti **non invia OK** (*l'altro processo non può andare avanti*) e mette il **sender** in una coda per tenere traccia dei processi a cui il processo dovrà poi inviare **OK** quando uscirà dalla CS.

Quindi prima o poi il **sender** riceverà **OK** e verrà sbloccato (**ME2**).

```
procedure onOk(proc sender, msg m)
  pending_oks = pending_oks-1
```

La funzione **onOk** viene eseguita ogni volta che viene ricevuto un **OK**, semplicemente decrementa il valore di **pending_oks** (*il numero di OK necessari ad entrare nella CS*).

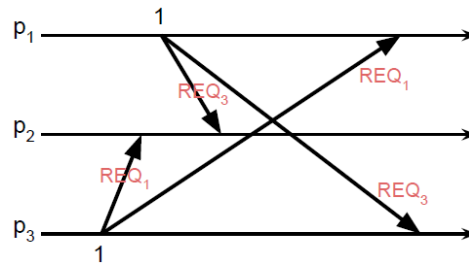
In uscita dalla CS il processo esegue:

```
procedure rel_access()
  mystate = OUT_CS
  foreach p in req_delayed: send(p, ok)
  req_delayed = empty
```

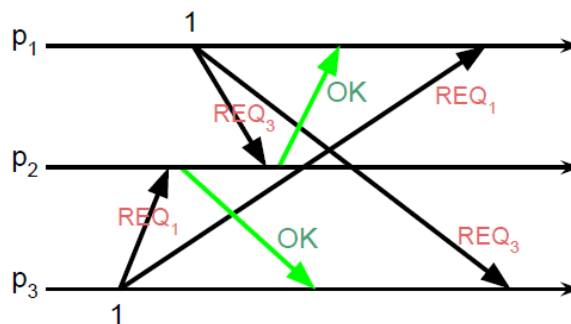
Dove invia **OK** a tutti i processi che ha messo in attesa precedentemente.

Esempio di esecuzione dell'algoritmo

In questo esempio osserviamo cosa succede se p_1 e p_3 provano ad entrare nella sezione critica in modo concorrente.



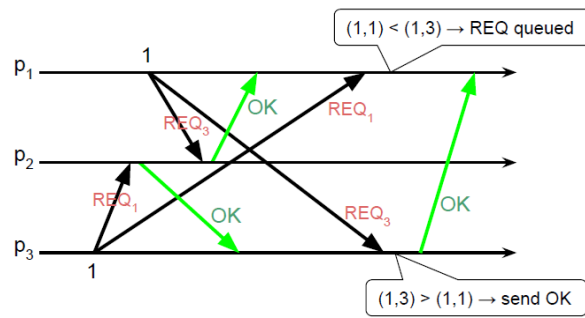
1. p_2 riceve REQ_1 da p_3 .
2. p_2 risponde con OK , perchè p_2 non è interessato nell'ingresso alla CS (OUT_CS).
3. p_2 riceve REQ_3 da p_1 .
4. p_2 risponde con OK , perchè p_2 non è interessato nell'ingresso alla CS (OUT_CS).



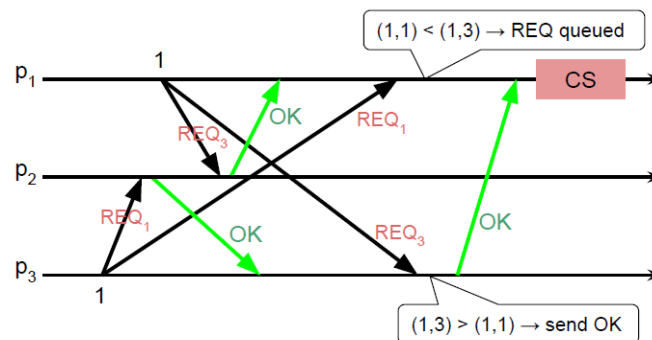
5. p_1 riceve REQ_1 da p_3 .
6. p_1 **non** risponde con OK , perché:
 - p_1 è interessato nell'ingresso alla CS (REQ_CS).
 - p_1 ha priorità maggiore $(1, 1) < (1, 3)$.

p_1 mette p_3 nella sua coda.

7. p_3 riceve REQ_3 da p_1 .
8. p_3 risponde con OK , perchè p_1 ha priorità maggiore $(1, 1) < (1, 3)$.

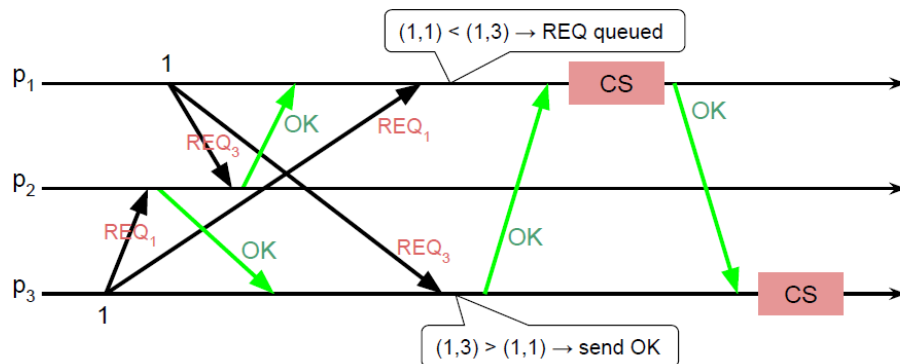


9. p_1 esegue la sezione critica (ha ricevuto i due OK che gli servivano).



10. p_1 termina l'esecuzione nella sezione critica, invia OK a p_3 .

11. p_3 può entrare nella sezione critica ed iniziare la sua esecuzione.



Complessità del numero di messaggi dell'algoritmo Ricart-Agrawala

Ogni ingresso nella sezione critica richiede:

- L'invio di $n - 1$ REQ .
- La ricezione di $n - 1$ OK .

Con n processi si avranno $2(n - 1)$ messaggi.

Complessità del tempo dell'algoritmo Ricart-Agrawala

Supponendo di avere uno **spanning tree** per il broadcast dei REQ e il converge cast degli OK.

Uno **spanning tree** è una struttura ad albero che connette tutti i nodi di una rete senza formare cicli e con il numero minimo di collegamenti possibile. In questo contesto, viene utilizzato per gestire due operazioni fondamentali:

1. Broadcast dei REQ (Request)

Il **broadcast** significa che un messaggio di richiesta (REQ) parte da un nodo sorgente e viene propagato a tutti gli altri nodi della rete. Usando uno spanning tree, il messaggio segue i rami dell'albero in modo efficiente, evitando duplicazioni o cicli inutili.

2. Convergecast degli OK (Acknowledgment o Risposte positive)

Il **convergecast** è il processo inverso del broadcast. I messaggi di risposta (OK) partono dai nodi foglia e risalgono l'albero verso la radice. Ogni nodo raccoglie le risposte dai propri figli e le inoltra verso il genitore, fino a raggiungere il nodo sorgente originale.

Nel caso peggiore il **costo in termini di tempo dipende dal percorso più lungo possibile (numero di hop)** tra due processi generici nella rete.

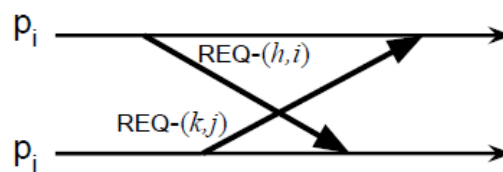
Quindi, dipende dal diametro D della rete (cioè il numero massimo di hop tra due nodi).

Complessità Temporale: $O(D)$.

ME1 - Correttezza dell'Algoritmo Ricart-Agrawala

Dimostriamo che l'algoritmo garantisce ME1: "al massimo un processo alla volta può essere nella sezione critica".

Scenario 1: I processi inviano la REQ prima di ricevere quella dell'altro.



Supponiamo che il timestamp della REQ di $p_i = (h, i)$ e quello della REQ di $p_j = (k, j)$.

Supponiamo che p_i abbia priorità maggiore a p_j , ovvero $p_i < p_j \Leftrightarrow (h, i) < (k, j)$.

1. p_j riceve REQ da p_i .

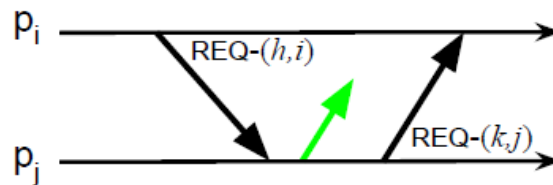
Secondo le regole dell'algoritmo, p_j confronta i due timestamp.

2. Poiché $(h, i) < (k, j)$, p_j invia OK a p_i .

3. Tuttavia, p_i riceve REQ di p_j .
4. Poiché $(h, i) < (k, j)$, p_i **non invia OK** a p_j , dato che p_i è già in attesa di entrare nella CS e ha la priorità.

p_j non ottiene OK $\rightarrow$ solo un processo è nella CS $\rightarrow$ ME1 verificata

Scenario 2: Un processo invia OK prima di inviare REQ.



Supponiamo che p_j abbia inviato OK a p_i prima di inviare la propria richiesta (REQ).

Supponiamo che p_i abbia priorità maggiore a p_j , ovvero $p_i < p_j \Leftrightarrow (h, i) < (k, j)$.

1. p_j riceve REQ di p_i con timestamp h , aggiorna il proprio clock logico
 $myclock_{(j)} = \max(myclock_{(j)}, h)$.

Questo garantisce che $myclock_{(j)} \geq h$.

2. p_j invia OK a $p_i \rightarrow p_i$ entra nella CS.
3. p_j invia REQ, il timestamp associato sarà $k = (myclock_{(j)} + 1) > h$.
4. p_i riceve REQ di $p_j \rightarrow p_i$ non invia OK, perché il proprio stato non è OUT_CS e ha priorità maggiore.

p_j non ottiene OK $\rightarrow$ solo un processo è nella CS $\rightarrow$ ME1 verificata

ME2 - Assenza di Deadlock e Starvation

L'algoritmo garantisce ME2 (**liveness**), ovvero:

- **Assenza di deadlock:** Non si possono verificare cicli di attese reciproche. Ogni richiesta viene processata e le risposte vengono inviate seguendo un ordine deterministico (*basato sui timestamp*).
- **Assenza di starvation:** Ogni processo, una volta inviata la propria richiesta, riceverà prima o poi tutte le risposte necessarie (OK) e potrà accedere alla CS. Questo è garantito dall'ordine totale imposto sui timestamp.

In sintesi, ogni processo attende al massimo il tempo necessario affinché tutti gli altri processi con richieste precedenti completino il loro ciclo nella CS.

In questo caso è importante rispettare dei pattern, perché se non vengono rispettati non è garantito che le cose vengano eseguite nel modo giusto.

Leader Election in un contesto in cui ci possono essere dei fault.

I fallimenti vengono gestiti secondo un modello e a seconda del modello, uno o più componenti ne sono affetti.

Fail-stop

Il modello più utilizzato.

Il processo si blocca e rimane bloccato. Gli altri processi possono rilevare questo stato.

Influenza un processo.

Crash

Il processo si blocca e rimane bloccato. Gli altri processi potrebbero non essere in grado di rilevare questo stato.

Influenza un processo.

Omission (canale)

Il messaggio viene perso.

Influenza un canale e anche il processo ricevente in caso non potesse gestire questo errore.

Byzantine

L'errore è gestito secondo comportamenti arbitrari.

Influenza un processo e il canale di comunicazione.

Algoritmi per l'elezione di un leader

I processi scelgono uno di loro che da ora in poi dovrà svolgere dei compiti speciali, spesso relativi alla coordinazione.

Ruoli durante l'elezione

Per scegliere un leader nel sistema, ogni processo p_i ha un identificatore unico **UID_i**.

Durante un'elezione, un processo può avere due ruoli:

- **Partecipante**: Coinvolto attivamente nell'elezione.
- **Non partecipante**: Non coinvolto o ha terminato la partecipazione.

Un processo in alcuni casi può avere le seguenti variabili:

- **leader_i**: Contiene l'identificatore del leader.
- **done_i**: Booleano per indicare se l'elezione è terminata.

Proprietà

- **LE1 (Safety)**:
 - Viene eletto al massimo un solo leader non guasto (*non-crashed*).
 - Gli stati **elected_i** e **done_i** rimangono stabili.
 - Alla terminazione, il leader è stato eletto.

- **LE2 (Liveness):**
 - Un processo verrà, prima o poi, eletto.
 - L'elezione è nota a tutti i processi non guasti (*non-crashed*).

Nota importante: È stato dimostrato che in un sistema anonimo (senza **UID**) e con una topologia regolare di rete, il problema dell'elezione del leader non è risolvibile.

Algoritmo di Chang & Roberts (C&R)

Presuppone una topologia ad **anello unidirezionale**, senza che i canali siano FIFO.

Il leader sarà il processo con **UID** massimo.

Funzionamento

1. L'elezione viene avviata da un initiator che invia un messaggio **ELECTION** lungo l'anello.
2. Ogni processo che vede arrivare un **ELECTION** invia a sua volta un **ELECTION** con il proprio **UID**.

Il messaggio **ELECTION** contiene lo **UID** del mittente.

3. Ogni processo blocca ogni **ELECTION** con un **UID** inferiore **al proprio** e inoltra quelli con **UID** maggiore.

Data la regola (2) solo lo **UID** massimo può completare un giro dell'anello e vincere.

4. Il processo che vede ritornare il proprio **ELECTION** capisce di essere il leader.

Non importa quanti initiator ci sono nel sistema, il leader stesso è quello che scopre di essere il leader.

5. Il leader invia un messaggio **LEADER** per informare gli altri.

```
procedure startElection()
  mystate = participant
  election.UID = myUID
  send(election)
```

startElection() è il metodo che consente di avviare il meccanismo di elezione, dobbiamo mantenere l'informazione mystate per capire se partecipiamo all'elezione oppure vogliamo inviare solo un messaggio.

```
procedure onElection(msg m)
  if myUID < m.UID then
    mystate = participant
    send(m)
  elif myUID > m.UID then
    if mystate == nonparticipant
    mystate = participant
    election.UID = myUID
    send(election)
  elif myUID == m.UID then
    leader.UID = myUID
    send(leader); elected = true
```

`onElection()` viene utilizzata per controllare se il processo è il leader o meno.

Nel metodo `onElection()` se il nostro `UID` è minore di un `UID` ricevuto allora passiamo l'elezione ad un nostro vicino. In caso contrario il nostro processo è un possibile leader, ci sono due casi possibili se non è un partecipante viene settato come partecipante.

Se il processo riceve un messaggio con il proprio `UID`, significa che quel messaggio che ha inviato ha completato il giro, quindi il processo è il leader, è l'unico che conosce di essere il leader, allora invia un `LEADER` message per informare gli altri che lui è il leader.

```
procedure onLeader(msg m)
  leader = m.UID; done = true
  if myUID != m.UID then
    elected = false
  send(m)
```

Se però il processo riceve un messaggio `LEADER` allora viene eseguita la `onLeader()`: se lo `UID` ricevuto è diverso dal proprio, invia il messaggio al vicino, altrimenti non fa nulla.

Ovvero appena il `LEADER` torna al leader il giro si ferma.

Complessità del numero di messaggi

Caso migliore:

Vi è un **unico initiator** che avvia l'elezione, ed è anche il processo con il `UID` più alto.

- `ELECTION` viaggia una sola volta intorno all'anello (*ogni processo lo riceve e lo inoltra*), richiedendo n messaggi.
- `LEADER` viaggia una sola volta intorno all'anello (*ogni processo lo riceve e lo inoltra*), richiedendo n messaggi.

Costo totale: $N_m = n + n \sim O(n)$.

Caso peggiore: Tutti i processi sono initiator e i `UID` sono ordinati in modo decrescente sull'anello.

- Il processo con il `UID` massimo invia n messaggi `ELECTION` (*uno per ciascun processo*)
- Quello con il secondo `UID` invia $n - 1$ messaggi (*il futuro leader è l'unico che non lo inoltra*)
- Quello con il terzo `UID` invia $n - 2$ messaggi (*il futuro leader e il precedente nodo sono gli unici non lo inoltrano*)
- ...
- L'ultimo processo che invia un solo messaggio.

Il numero di `ELECTION` è: $n + (n - 1) + (n - 2) + \dots + 1 = \frac{n(n+1)}{2}$

Successivamente, il leader invia n messaggi **LEADER**.

Costo totale: $O(n^2)$.

Caso medio:

L'elezione viene avviata da un sottoinsieme di processi in modo casuale, la complessità media è $O(n \log n)$.

Questa stima si basa sull'analisi probabilistica del numero di messaggi **ELECTION** e **LEADER**, che dipende dalla distribuzione degli **UID** e dal numero di iniziatori.

Complessità del tempo

Caso migliore

Un unico initiator è quello con il **UID** più alto invia un solo giro di messaggi **ELECTION** intorno all'anello.

Questo richiede $O(n)$ in termini di tempo (*una comunicazione da processo a processo su un anello*).

Caso peggiore

Initiator è il processo con il secondo **UID** più alto e si trova immediatamente prima del processo con il **UID** massimo nell'anello (quindi sarà l'ultimo a ricevere **ELECTION**).

Il processo con il secondo **UID** più alto invia $n - 1$ messaggi **ELECTION** e poi, quando il messaggio raggiunge il processo con il **UID** massimo, quest'ultimo invia un altro giro di n messaggi.

Tempo totale: $(n - 1) + n \sim O(n)$.

Gestione dei crash durante l'elezione

Quando i processi possono fallire, l'algoritmo deve gestire in modo robusto queste situazioni, garantendo comunque che il leader venga eletto.

Ogni processo ha una forma differente del problema, il problema non consiste nel trovare chi ha **UID** maggiore, ma di trovare quale processo correntemente attivo ha lo **UID** più elevato.

Ipotesi:

1. **Consegna affidabile dei messaggi:**
 - Ogni messaggio inviato da un processo corretto viene consegnato.
2. **Sistema sincrono:**
 - Esiste un limite superiore noto sul tempo di latenza dei messaggi.
3. **Rilevamento dei guasti:**

- I fallimenti dei processi possono essere rilevati tramite meccanismi come i **timeout**.

Presupposti:

1. Ogni processo conosce gli **UID** degli altri processi. Ogni nodo fa sapere il proprio **UID** agli altri.
2. Ogni processo può comunicare con gli altri.
3. Il leader deve essere il processo con il **UID** più alto tra i processi attivi (*cioè non guasti*).

Problematiche principali:

1. **Processi guasti durante l'elezione:**
 - Se un processo si arresta mentre inoltra un messaggio, è necessario che i processi rimanenti rilevino il fallimento e riprendano l'elezione.
2. **Dipendenza dalla conoscenza dei **UID**:**
 - Per garantire che il leader sia il processo con il **UID** massimo tra i processi attivi, è necessario mantenere una mappatura aggiornata dei processi attivi (*quelli che non sono falliti*).

Gestione dei crash:

- Quando un processo rileva che il leader corrente è guasto (*ad esempio tramite un timeout*), può:
 1. Avviare una nuova elezione inviando un messaggio **ELECTION** agli altri processi.
 2. Se altri processi rispondono, seguirà la logica dell'algoritmo per eleggere il processo con il **UID** più alto tra quelli attivi.
 3. Se non riceve risposte, si autoproclama leader.

In base all'istante di tempo, il leader può essere differente a causa dei crash.

Caratteristica	C&R Standard	C&R con Crash
Robustezza ai guasti	Non prevista	Rilevamento dei guasti con timeout
Presupposti	Topologia ad anello	Topologia più generale, processi connessi
Leader scelto	UID massimo	UID massimo tra i processi attivi
Complessità messaggi	$O(n)$ (caso migliore), $O(n^2)$ (caso peggiore)	Dipende dai fallimenti (<i>può aumentare</i>)
Sicurezza (LE1)	Garantita	Garantita

Vitalità (<i>LE2</i>)	Garantita	Garantita (<i>se il sistema non è troppo guasto</i>)
------------------------------	-----------	--

- **The bully's algorithm**

L'algoritmo del Bullo è una tecnica di **elezione di un leader** in sistemi distribuiti.

Viene usato per scegliere un processo coordinatore tra un gruppo di processi, anche in presenza di fallimenti.

Si basa sull'assunzione che ogni processo abbia un **UID** unico e che i processi conoscano gli **UID** di tutti gli altri.

Tipi di messaggi

L'algoritmo utilizza tre tipi principali di messaggi:

- **ELECTION:**
 - Serve per annunciare l'avvio di un'elezione e viene inviato da un qualsiasi processo initiator.

Una nuova elezione inizia quando il leader corrente viene decretato fallito.

- **ANSWER:**
 - È una risposta a un messaggio **ELECTION**.
 - Indica che il destinatario è vivo e disponibile a partecipare al processo di elezione.
- **LEADER:**
 - Serve per annunciare che un processo è stato eletto leader.

Meccanismo di base

1. Un processo p rileva che il leader corrente è inaccessibile (*ad esempio, tramite un timeout*).

Timeout: se il leader non risponde dopo un certo intervallo, allora è fallito.

2. p invia **ELECTION** a tutti i processi con **UID superiore al proprio**.
 - I processi con **UID** maggiore rispondono con **ANSWER**.
 - I processi con **UID** minore saranno informati tramite un altro messaggio.

I processi con **UID** maggiore, dopo aver risposto, avviano un'elezione propria (*partono dal punto 1*).

3. Se p **riceve ANSWER**, attende per un tempo T' aggiuntivo per un **LEADER** proveniente dal nuovo leader.
 - Se T' scade, p avvia un nuovo turno di elezione (*si riparte dal punto 1*).
4. Se p **non riceve ANSWER** entro un certo intervallo di tempo T , significa che nessun processo con **UID** superiore al suo è attivo (tolleranza ai guasti) e quindi lui sarà il leader.

- p si proclama leader e invia **LEADER** a tutti i processi con **UID** inferiore.
5. Quando un processo riceve un messaggio **LEADER** da un processo con **UID** superiore, aggiorna la propria variabile locale **leader** per indicare il nuovo leader.

Scalabilità e semplicità

Inviare il messaggio solo ai processi con **UID** superiore riduce il numero complessivo di messaggi rispetto a un approccio in cui ogni processo lo invia a tutti gli altri.

Comportamento da bullo:

Non importa che il leader corrente sia ancora attivo, quando un nuovo processo si avvia e possiede il **UID** più alto, può decidere di autoproclamarsi leader senza verificare lo stato del leader attuale.

Questo comportamento "aggressivo" ha ispirato il nome dell'algoritmo.

Proprietà dell'algoritmo - Safety (LE1)

Non possono esistere due leader attivi contemporaneamente, perché i processi con **UID** inferiore cedono sempre al leader con **UID** superiore.

Proprietà dell'algoritmo - Liveness (LE2)

L'algoritmo garantisce che un leader venga sempre eletto, purché i messaggi vengano consegnati correttamente e il sistema non sia completamente guasto.

Esempio di esecuzione dell'algoritmo

In questo esempio i cerchi rappresentano dei processi, quelli **verdi** sono attivi, quelli **grigi** no.

Supponiamo che inizialmente il **processo grigio** fosse il leader, però dopo un certo tempo, il **primo processo verde** contatta quello **grigio** e scopre che non è più attivo.



Allora il primo processo contatta solo i processi con **UID** maggiore del proprio (*il secondo ed il terzo*) inviando **ELECTION**.



Il secondo ed il terzo processo rispondono al primo con un messaggio **ANSWER**.



Ma questi due processi non sanno chi è attivo e chi no.

Allora inviano i messaggi **ELECTION** ai processi alla loro destra (*terzo e quarto*) per capire chi è il leader.



Il terzo processo non riceve risposta dal quarto, mentre il secondo riceve un **ANSWER** dal terzo.



A questo punto il terzo processo è il leader (*non ha ricevuto nessun ANSWER*), quindi vengono informati gli altri processi attivi (*primo e secondo*) che lui è il leader con un **LEADER**.



Quando il processo inattivo torna attivo, non conosce chi è il leader, quindi invierà un **ELECTION** e inizierà una nuova elezione.

Complessità - Numero di messaggi

Caso migliore: L'initiator è il processo con il **UID** più alto.

Questo processo invia $n - 1$ messaggi **LEADER** per notificare il proprio ruolo a tutti gli altri processi. Non abbiamo messaggi **ELECTION**.

Complessità: $N_m = n - 1 \sim O(n)$.

Caso peggiore: L'initiator è il processo con il **UID** più basso.

Deve inviare un **ELECTION** a tutti i processi con **UID** superiore, quindi $N_{E1} = n - 1$.

Ogni processo superiore (tutti gli altri) avvia a sua volta un'elezione:

$$N_{E2} = n - 2 + N_{E3} = n - 3 + \dots + N_{En} = 0$$

Complessità: $N_m \sim O(n^2)$

Complessità - Tempo

Caso migliore: L'initiator è il processo con il **UID** più alto.

L'elezione richiede un solo giro di messaggi **LEADER**, completandosi in $O(n)$.

Caso peggiore: L'initiator è il processo con il **UID** più basso.

Richiede più round di comunicazione per propagare i messaggi **ELECTION** e aspettare risposte, completandosi in $O(n^2)$.

Vantaggi e svantaggi

- **Semplice** da implementare in sistemi distribuiti.
- Garantisce **LE1** e **LE2**.
- Supporta il **rilevamento di guasti** tramite timeout.
- **Inefficiente con molti processi**, a causa della complessità $O(n^2)$ nel caso peggiore.
- La strategia "da bullo" può causare **elezioni non necessarie**, aumentando il carico di rete.

CAPITOLO 13 - SYNCHRONIZATION OF PHYSICAL CLOCKS

- **Synchronization of physical clocks, general principles**

Ogni nodo di un sistema distribuito possiede un proprio **clock interno**, che deve essere sincronizzato il più possibile con un **tempo di riferimento**.

Esistono diversi standard di riferimento temporale:

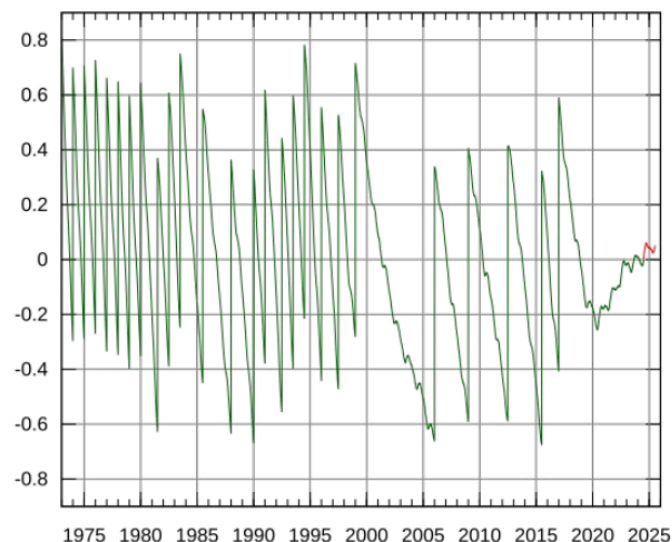
- **UTC (*Coordinated Universal Time*)**: è il riferimento principale per la comunità scientifica, basato sul **Tempo Atomico Internazionale**. Non è collegato a nessun riferimento astronomico.
- **UT1**: Successore del Greenwich Mean Time (*GMT*), è un riferimento "basato sul moto solare".
- **Unix (o *POSIX*) time**: rappresenta il numero di secondi trascorsi dalle 00:00:00 UTC del 1° gennaio 1970, esclusi i secondi intercalari (*leap seconds*).

UTC - Leap Seconds

A causa del rallentamento e delle irregolarità della rotazione terrestre, UT1 non è del tutto affidabile: UT1 si discosta dal tempo UTC.

Per questo motivo, UTC ha bisogno di alcune correzioni.

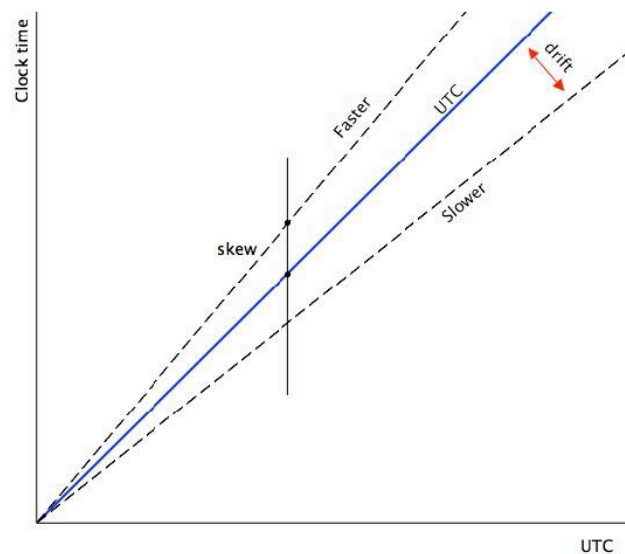
Per correggere questa discrepanza, vengono introdotti i **secondi intercalari**, indicati come segmenti verticali nei grafici temporali.



Disallineamento dei Clock

Due clock possono differire in due modi:

- **Clock Skew:** differenza istantanea tra i valori letti da due clock (due timer partiti in due momenti diversi).
- **Clock Drift:** differenza nella velocità (tasso) di avanzamento di due clock (due timer che avanzano con due velocità diverse).



Diremo che due clock sono sincronizzati solo se il loro clock skew è minore di un certo valore D .

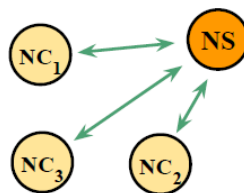
Supponiamo che il nostro clock locale sia la linea tratteggiata.

Sincronizzazione esterna

Ogni nodo si sincronizza rispetto a una **fonte di riferimento affidabile** $S(t)$, assicurando:

$$|S(t) - C_i(t)| < D, \forall NC_i$$

In tal caso si dice che i nodi sono sincronizzati esternamente con accuratezza D .

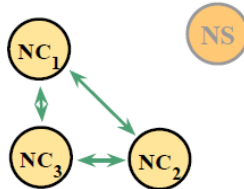


Sincronizzazione interna

I **nodi di un gruppo** concordano tra loro in modo che:

$$|C_i(t) - C_j(t)| < D, \forall NC_i, NC_j$$

In tal caso si dice che i nodi sono sincronizzati internamente con accuratezza D .



Relazione tra sincronizzazione esterna e interna

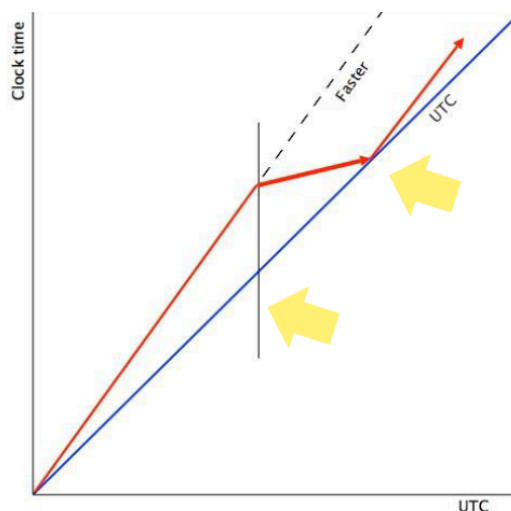
Se un sistema è sincronizzato esternamente con accuratezza D , allora i suoi clock interni concordano con una differenza massima di $2D$.

Ripristino di un Clock

Quando si corregge un valore per un clock, è essenziale garantire la **monotonicità temporale** (cioè, il tempo non deve mai "tornare indietro nel tempo").

Perciò, anziché impostare indietro il clock (ovvero *diminuire il clock skew*), si rallenta il suo ritmo (*diminuiamo il clock drift*) fino a raggiungere il valore corretto.

Se il nostro clock è troppo veloce, come in figura, e ad un certo tempo vediamo che il valore corretto dovrebbe essere un altro, non è possibile correggerlo immediatamente perchè non è corretto portare il clock indietro, quindi viene rallentato (o in alcuni casi fermato) fino ad ottenere il valore corretto.



- Se io sono su una rete locale e devo sincronizzare i clock dei singoli nodi, come posso fare? Illustra i vari protocolli.

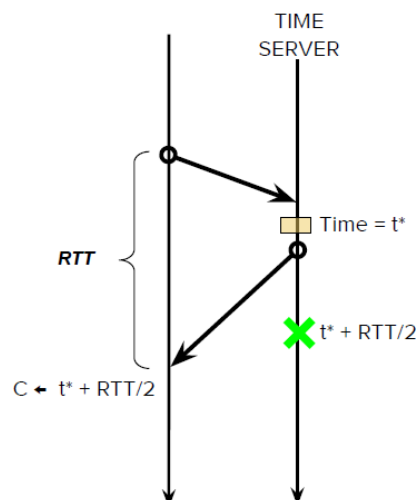
Risposta: Non è possibile avere un global clock

- La richiesta specifica è che siano tra di loro sincronizzati (secondo me si potrebbe avere un server nella rete locale)

Sincr. esterna - Algoritmo dei Cristiani †

L'algoritmo sfrutta il **Round Trip Time (RTT)**, cioè il tempo di andata e ritorno di un messaggio tra client e server.

1. Un client invia una richiesta di sincronizzazione al server di riferimento.
2. Il server risponde con il proprio timestamp attuale.
3. Il client stima il tempo di rete (*basandosi sullo RTT*) e corregge il proprio orologio **assumendo che la latenza sia simmetrica** (ovvero il tempo di andata è uguale a quello del ritorno).



Latenza simmetrica

Se RTT è molto piccolo ($\ll 1$), si può assumere che i tempi di latenza per la richiesta e la risposta siano simmetrici.

Calcolo lato client

Se il server inviasse il tempo corrente misurato al momento di quando riceve la risposta, il client otterrebbe un valore non aggiornato.

Sia t^* il timestamp attuale del server.

In tal caso, il client può misurare solo RTT, quindi calcola il tempo corretto in questo modo:

$$t_{client} \leftarrow t^* + \frac{RTT}{2}$$

Più basso è lo RTT e più precisa sarà la sincronizzazione, per questo conviene fare più tentativi per ottenere un RTT più basso.

- In pratica, quale è il massimo dell'errore che posso permettere nel sincronizzare i clock?

Risposta: L'errore massimo è nel caso in cui l'invio è immediato e tutto il RTT è dovuto all'invio/ricezione del secondo pacchetto, quindi il massimo errore è $RTT/2$

- Algoritmo di Berkeley, quale problema risolve? Descrivere approfonditamente

Sincr interna - Algoritmo di Berkeley

Soluzione **centralizzata** progettata per gruppi di computer UNIX:

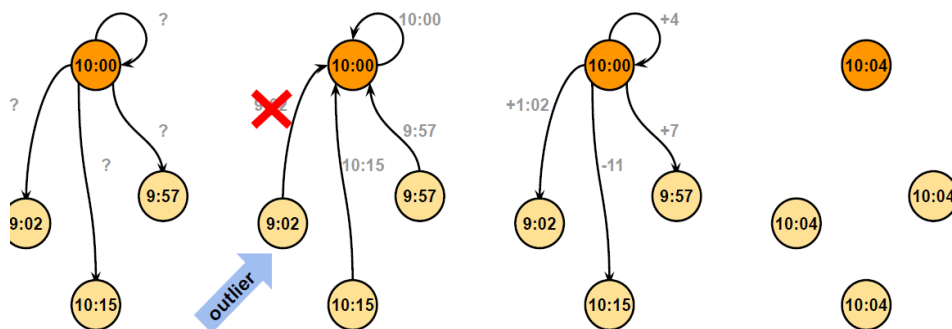
1. Un processo centrale (**time daemon**) opera come **master** e raccoglie i valori dei clock dagli altri nodi.
2. Ogni nodo invia il proprio valore temporale; il time daemon annota i tempi insieme anche ai rispettivi RTT (come nell'algoritmo dei cristiani).
3. Il time daemon calcola una media dei valori raccolti.
4. Il time daemon invia a ogni nodo la correzione che deve essere applicata.

Perché nel punto 4 non viene inviato il tempo in broadcast?

Perché arriverebbe ad ogni nodo in punti differenti nel tempo (anche nel punto 4 vi è un messaggio e lo RTT può differire di molto dall'altro), quindi ci sarebbero clock skew differenti per ogni nodo.

Inviando solo il valore di correzione, il reset del clock avviene in modo time-insensitive, in base a quando ogni nodo riceve la correzione.

eventuali outlier non vengono considerati nel calcolo della media, ma gli sarà comunque inviata una correzione per loro.



Applicare correzioni negative non si può fare, quindi il nodo che riceve una correzione negativa (significa che il suo clock è troppo avanti nel tempo) applicherà una delle tecniche descritte in precedenza (lo rallenta fino al valore desiderato).

L'offset di ogni nodo è calcolato in base allo RTT percepito da ogni nodo.

Network Time Protocol

Permette ai client di sincronizzarsi con UTC attraverso Internet:

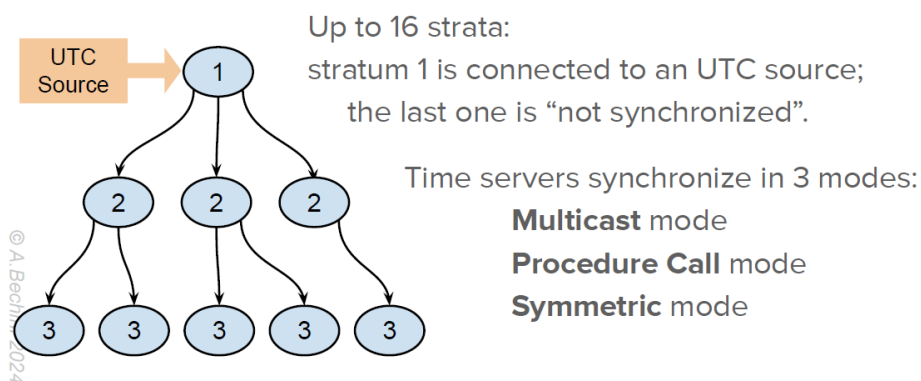
- **Scalabilità** per grandi reti, perché bisogna considerare ogni nodo su internet.
- Tecniche statistiche per filtrare i dati.
- **Autenticazione** per evitare problemi relativi ad interferenze.
- **Gerarchia di server temporali** distribuiti su Internet.
- **UDP** per inviare i messaggi.

Gerarchia di Server Temporali (Strati)

I server NTP sono organizzati in **strati (strata)**, fino a un massimo di 16:

- Lo strato 1 è direttamente connesso a una fonte UTC.
- Ogni server di uno strato è sincronizzato (*approccio client-server, come l'algoritmo dei cristiani*) con un server dello strato precedente.
- Lo strato finale è considerato "*non sincronizzato*".

Più un nodo è vicino alla sorgente UTC ufficiale, più sincronizzato è; perché gli algoritmi non sono perfetti e quindi più ci si allontana e più impreciso diventa il riferimento.



Modalità di Sincronizzazione in NTP

NTP offre tre modalità principali di sincronizzazione tra server e nodi:

Modalità Multicast

Uno o più server inviano messaggi di sincronizzazione (*time multicast*) a più nodi della rete.

Non vengono utilizzate correzioni message delay, quindi è particolarmente adatta a reti locali (LAN), dove gli RTT sono molto piccoli e poco variabili.

I nodi che ricevono il multicast aggiornano (sostituiscono il valore) i propri clock basandosi sull'ora ricevuta dal server.

In questa modalità si presuppone un RTT molto basso e variabile, quindi non viene applicata una precisa compensazione delle latenze, rendendola meno accurata rispetto ad altre modalità.

Modalità Procedure Call

Più accurata rispetto al multicast, poiché considera la **compensazione delle latenze**, ma rimane comunque molto efficiente.

Un client invia una richiesta al server basandosi sul **Christian's algorithm**.

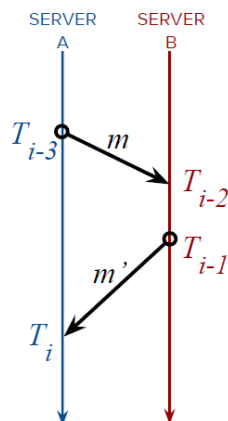
Questa modalità è più adatta per ambienti in cui si richiede una buona accuratezza senza un costo computazionale eccessivo.

Modalità Simmetrica

La modalità **più precisa**, ma anche la **più costosa** in termini di risorse.

Viene utilizzata principalmente tra server appartenenti agli strati più alti della gerarchia NTP, dove è essenziale una sincronizzazione estremamente accurata (*per evitare di propagare l'errore agli strati successivi*).

Una **coppia di server** si scambiano due messaggi contenenti i timestamp al momento dell'invio.



Ognuno dei due server ha quindi i 4 timestamp.

Poi stimano lo **scostamento reale** (offset o) tra i propri clock. Lo scostamento stimato (o_i) si calcola con la formula:

$$o_i = \frac{1}{2} (T_{i-2} - T_{i-3} + T_{i-1} - T_i)$$

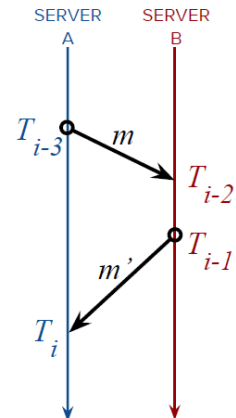
Quanto è precisa questa formula? Ovvero vediamo l'errore commesso a quanto ammonta.

Dal grafico sopra vediamo bene a quanto ammontano queste due grandezze:

$$\begin{cases} T_{i-2} = T_{i-3} + t_m + o \\ T_i = T_{i-1} + t_{m'} - o \end{cases}$$

Quindi l'offset è calcolabile come:

$$\begin{cases} o = T_{i-2} - T_{i-3} - t_m \\ o = T_i - T_{i-1} + t_{m'} \end{cases}$$



Possiamo esprimere RTT in modo certo come il tempo in cui i due server non hanno lavorato.

$$RTT = t_m + t_{m'} = \Delta T_A - \Delta T_B = (T_i - T_{i-3}) - (T_{i-1} - T_{i-2})$$

$$o = \frac{1}{2} (T_{i-2} - T_{i-3} + T_{i-1} - T_i - t_m + t_{m'}) = o_i + \frac{1}{2} (t_{m'} - t_m)$$

Infine:

$$o_i - \frac{1}{2} RTT \leq o \leq o_i + \frac{1}{2} RTT$$

Consideriamo i casi estremi:

- Caso 1 (ritorno istantaneo, andata lenta): $t_{m'} = 0, t_m = RTT$
- Caso 2 (andata istantanea, ritorno lento): $t_m = 0, t_{m'} = RTT$

In questi casi estremi, l'offset reale o può variare da:

$$o = o_i - \frac{1}{2} RTT \quad (\text{quando } t_{m'} < t_m)$$

fino a:

$$o = o_i + \frac{1}{2} RTT \quad (\text{quando } t_{m'} > t_m)$$

Correzione del Tempo

Ovviamente, più basso è RTT e più precisa è la misurazione, quindi conviene fare molteplici misurazioni usando quella con RTT minore.

Per correggere il tempo, i server raccolgono più coppie di valori $\langle o_i, RTT \rangle$. Ogni coppia include:

Tra tutte le coppie raccolte, viene scelto il valore che garantisce la maggiore accuratezza, ossia quello con il **minore RTT** o quello statisticamente più affidabile.